Stock Price Prediction using Algorithmic Trading

A Hybrid Sentiment-Aware Stock Prediction Study

Submitted by

Anik Das
Agnirudra Banerjee
Mohar Mukherjee
Bibhas Roy

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

Sister Nivedita University
Newtown, Kolkata, West Bengal

2025

Stock Price Prediction using Algorithmic Trading

Project Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology (CSE)

Name	Enrollment	Email ID
Agnirudra Banerjee	2211200010029	agnirudrabanerjee777@gmail.com
Bibhas Roy	2211200010001	abc.3gbibhas@gmail.com
Mohar Mukherjee	2211200010010	MOHARMUKHERJEE2004@gmail.com
Anik Das	2211200010038	anik200365@gmail.com

Submission Date: 21-November, 2025

Under the supervision of

Dr. Soma Datta

Assistant Professor

Department of Computer Science

Sister Nivedita University, Newtown

Kolkata, West Bengal

Sister Nivedita University

Newtown, Kolkata, West Bengal

November, 2025

DECLARATION

We hereby declare that the project work entitled **"Stock Price Prediction using Algorithmic Trading"** submitted to the Department of Computer Science, Sister Nivedita University, is a record of an original work done by us under the supervision of **Dr. Soma Datta**, Assistant Professor, Department of Computer Science, Sister Nivedita University.

This project work is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

.....	.....
Agnirudra Banerjee	**Bibhas Roy**
.....	.....
Mohar Mukherjee	**Anik Das**

Place: Kolkata
Date: November, 2025

CERTIFICATE

This is to certify that the project report entitled **"Stock Price Prediction using Algorithmic Trading"** submitted by:

Agnirudra Banerjee	(ID: 2211200010029)
Bibhas Roy	(ID: 2211200010001)
Mohar Mukherjee	(ID: 2211200010010)
Anik Das	(ID: 2211200010038)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of Sister Nivedita University, is a bona fide record of the work carried out under my supervision and guidance.

Dr. Soma Datta
Assistant Professor
Department of Computer Science
Sister Nivedita University, Newtown, Kolkata

Contents

List of Figures

List of Tables

ACKNOWLEDGEMENT

We would like to express our deep sense of gratitude and profound respect to our supervisor, **Dr. Soma Datta**, Assistant Professor, Department of Computer Science, Sister Nivedita University, for her valuable guidance, constant encouragement, and constructive criticism throughout the course of this project work. Her expertise and insight have been invaluable in shaping our understanding of the subject.

We are also grateful to the **Department of Computer Science** and **Sister Nivedita University** for providing us with the necessary facilities and an environment conducive to learning and research.

Finally, we would like to thank our families and friends for their unwavering support and encouragement during this endeavor.

....
Agnirudra Banerjee **Bibhas Roy**

....
Mohar Mukherjee **Anik Das**

ABSTRACT

The prediction of stock market prices is a challenging task due to the non-linear, volatile, and sentiment-driven nature of financial time series data. Traditional statistical models like ARIMA have limitations in capturing non-linear patterns, while deep learning models like LSTM excel at sequential data but may struggle with noise. Price-only models, regardless of architectural sophistication, are structurally blind to exogenous information shocks—such as earnings surprises, macroeconomic announcements, and shifts in public sentiment—until those shocks are already reflected in historical prices. This literature review comprehensively explores the evolution of stock prediction methodologies, with a specific focus on **Hybrid Models** that combine the strengths of multiple approaches (e.g., ARIMA-LSTM, CNN-LSTM), and extends the analysis to **Sentiment-Aware Hybrid Architectures** that integrate financial news sentiment into the prediction pipeline.

We analyze various hybrid architectures, comparing their mechanisms, strengths, and limitations. The review highlights how hybrid models consistently outperform standalone models by effectively handling both linear and non-linear components, as well as incorporating diverse data sources such as technical indicators and macroeconomic variables. Building on this foundation, we propose a sentiment-augmented extension of the SARIMA-XGBoost hybrid model that integrates a domain-adapted sentiment analysis pipeline featuring dual-head sarcasm correction, principled temporal alignment of news publication timestamps to trading days, and confidence-weighted sentiment aggregation. The system addresses critical gaps identified in modern sentiment analysis literature—including sarcasm and irony detection, tokenization drift in financial language, domain sensitivity, and uncertainty quantification—and feeds structured sentiment features into the XGBoost residual learning stage, enabling the model to capture linear trends, non-linear residual patterns, and sentiment-driven price dynamics within a unified three-component decomposition $y_t = L_t + N_t + E_t + \epsilon_t$. Furthermore, this document identifies research gaps in current methodologies, particularly regarding multimodal data integration, the lack of adaptive learning mechanisms for regime shifts, and the absence of robust risk-aware prediction frameworks. These identified gaps and the proposed sentiment-aware architecture lay the foundation for more robust and accurate stock prediction systems.

1 Introduction

1.1 Context of the Study

The stock market is a core component of modern financial systems and a major channel for capital allocation. Predicting stock prices is difficult because market behavior is non-linear, noisy, and regime-dependent. Price dynamics usually reflect many interacting factors, including companies performance, macroeconomic conditions, big events, liquidity, and investor behavior.

Traditional statistical models are easy to understand and have solid theory behind them, but they can have trouble keeping up when markets change fast or things get complicated. Machine learning and deep learning are better at spotting patterns, but if they only look at price, they still miss out on important outside signals.

But even the smartest price-based models, whether they're statistical, machine learning, or deep learning, have a basic problem: they only look at past price and volume data, plus technical indicators. They can't react to sudden news or surprises, like a bad earnings report, a CEO stepping down, or a major policy change, until that information is reflected in the price. The reality is, financial markets run on information, news, earnings releases, analyst takes, and what people are saying all help set prices. If a model can actually pull out and measure the mood in financial news and use it in its predictions, it basically gets a head start that price-only models just don't have.

Integrating sentiment analysis into financial prediction is not straightforward. Modern sentiment models suffer from several well-documented gaps: financial language contains domain-specific jargon, hedged expressions, and sarcasm that general-purpose sentiment models misinterpret; news publication timestamps do not directly align with trading day boundaries, creating temporal mismatch problems; most sentiment pipelines produce deterministic point estimates with no uncertainty quantification; and models trained on general corpora experience tokenization drift when applied to financial text. This work is therefore motivated by the convergence of two research problems: the need for sentiment-aware trading models, and the need for robust, domain-aware sentiment analysis that addresses the gaps documented in existing literature.

1.2 Objectives

This literature review aims to:

- Study the evolution of stock prediction techniques from traditional statistical approaches to modern hybrid deep learning models.
- Compare statistical, machine learning, deep learning, and hybrid systems using stock-domain evidence.
- Identify research gaps, especially in sentiment integration, temporal alignment, and robust out-of-sample evaluation.

- Define a clear methodological basis for a hybrid SARIMA-XGBoost framework.
- Position a sentiment-augmented extension as a research direction for improving directional forecasting.

1.3 Scope and Organization

This report is organized into five chapters. Chapter 2 presents a critical stock-domain literature review. Chapter 3 defines the precise research gap and study objectives. Chapter 4 details the end-to-end methodology, including sentiment feature construction and model design. Chapter 5 reports implemented results for the baseline system and clearly separates proposed sentiment-extension evaluation from completed experiments.

1.4 Contributions

The contributions of this work are summarized as follows:

1. **Stock-Focused Literature Review.** The literature review is organized around stock-market evidence, with detailed comparison by model family, evaluation protocol, and deployment limitations.
2. **Hybrid Forecasting Approach.** The method uses a residual-learning hybrid pipeline that models trends with SARIMA and captures non-linear patterns with XGBoost.
3. **Sentiment Integration Blueprint.** A structured plan is provided for sentiment augmentation through domain-adapted scoring, sarcasm handling, and leakage-safe temporal assignment.
4. **Reproducible Evaluation Design.** The report specifies temporal splits, walk-forward principles, and both error-based and trading-relevant metrics for fair comparison.

2 Literature Review

2.1 Introduction to Literature Survey

This chapter reviews stock-market forecasting literature across statistical, machine learning, deep learning, and sentiment-aware hybrid methods. The review prioritizes studies that evaluate stock indices or listed equities directly, and reports evidence by market, dataset scope, evaluation metrics, and practical limitations. The objective is not to restate model definitions, but to identify what has been shown to work, where results conflict, and which gaps remain unresolved for real deployment.

2.2 Historical Approaches

2.2.1 Statistical Foundations

Early stock-forecasting studies relied on linear time-series modeling. ARIMA-style modeling remains a standard baseline because of interpretability and transparent diagnostics [1]. Volatility-focused modeling evolved through ARCH and GARCH, which are still foundational in risk and variance forecasting for financial series [2, 3]. These methods are valuable as reference models, but their assumptions limit performance under abrupt structural breaks and strong non-linearity.

2.2.2 Transition to Machine Learning

Machine learning introduced non-linear decision boundaries and feature interactions beyond linear econometric assumptions. SVM-based forecasting [4] and later ensemble/tree methods in equity-index settings [5, 6] showed that technical features and robust preprocessing can outperform basic linear baselines in direction classification tasks. However, performance varies by market regime, horizon, and feature construction, making reproducibility and protocol design central concerns.

2.2.3 Deep Learning Era

Sequence models such as LSTM became prominent because they model temporal dependencies without fully manual lag engineering [7]. In stock-domain evaluations, LSTM-class models frequently improve directional performance relative to shallow baselines, especially when feature windows are tuned and leakage is controlled [8, 9]. At the same time, gains are not uniform across assets: data quality, horizon choice, and market regime materially affect results.

2.2.4 From Sequence Models to Attention and Multimodal Systems

Attention-based architectures, motivated by transformer modeling [10], enabled richer long-context learning and multimodal fusion strategies. In financial prediction, event-driven text-price modeling demonstrated measurable improvements when textual events were aligned with

market movements [11]. This shift established a key design principle: financial forecasting quality depends on both temporal modeling and information-source alignment.

2.3 Existing Systems / Related Work

2.3.1 Traditional Statistical Methods

ARIMA/SARIMA remain strong baselines for trend and seasonality decomposition when assumptions are approximately satisfied [1]. Their strengths are interpretability, low compute cost, and stable diagnostics. Their limitations are well-known in equity forecasting: weak handling of non-linear cross-feature effects, sensitivity to structural breaks, and reduced adaptability under rapid regime transition.

2.3.2 Machine Learning Approaches

SVM, random forests, and gradient-boosting systems are widely used for stock direction prediction and residual correction tasks [4–6]. Across studies, model quality is heavily conditioned on feature design (lags, technical indicators, macro context) and temporal validation protocol. A recurring finding is that strong preprocessing and feature engineering can narrow or exceed the gap to more complex deep models in certain horizons.

2.3.3 Deep Learning-Based Approaches

LSTM/GRU/BiLSTM families are the most common deep architectures in financial forecasting benchmarks [9]. In stock-specific tests, LSTM-based systems often provide better sequence modeling than static learners, especially for short-horizon direction tasks [8]. Still, deep models can overfit in non-stationary markets and require careful walk-forward evaluation, retraining policy, and latency-aware deployment constraints.

2.3.4 Sentiment-Based Methods

Sentiment modeling evolved from dictionary approaches (e.g., domain lexicons) to transformer-based contextual models. Financial-domain language resources [12,13] and social-media market-signal studies [14] established that textual information can carry predictive value under specific settings. Domain-adapted transformers such as FinBERT [15], built on BERT [16], improved finance-text polarity classification; however, production use still depends on timestamp alignment, confidence handling, and drift control.

2.4 Hybrid Architectures

2.4.1 Rationale for Hybrid Design

Financial series exhibit mixed structure: a partly linear backbone plus non-linear residual behavior. Hybrid pipelines use this decomposition operationally by assigning trend/seasonal structure to statistical models and residual non-linearity to machine-learning learners.

2.4.2 Price-Only Hybrid Systems

ARIMA/SARIMA plus tree-boosting or neural residual correction is a common design pattern in stock prediction studies and industrial prototypes. Survey evidence indicates that hybrid and ensemble families are frequently more robust than single-model pipelines under heterogeneous market conditions [6, 9]. Practical success depends on leakage-safe residual construction and consistent out-of-sample testing.

2.4.3 Price + Sentiment Hybrid Systems

Recent systems extend hybrid pipelines with sentiment features from financial news and event streams. Event-driven deep models [11] and sentiment-market evidence [14] motivate this integration, while financial-domain NLP resources [12, 15] improve feature quality. The key unresolved engineering issue is not only sentiment scoring accuracy, but robust alignment between asynchronous text events and tradable market time.

2.5 Comparative Evidence from Literature

Table 1: Representative Evidence in Stock-Market Forecasting Literature

Study	**Market / Data**	**Method Family**	**Typical Metrics**	**Main Limitation**
Kim (2003) [4]	Equity index direction data	SVM classifier	Directional accuracy	Sensitive to feature specification and regime shifts
Patel et al. (2015) [5]	Indian equities + indices	ML comparison (ANN/SVM/RF/NB)	Accuracy, precision/recall	Preprocessing-dependent gains
Fischer & Krauss (2018) [8]	S&P 500 constituents	LSTM vs classical baselines	Accuracy, return simulation	Stability across regimes remains challenging
Sezer et al. (2020) [9]	Multi-study review (2005–2019)	Deep-learning survey	Cross-study comparison	Strong protocol heterogeneity
Nti et al. (2020) [6]	Cross-market stock studies	Ensemble evaluation	Accuracy-based measures	Limited risk-aware reporting
Ding et al. (2015) [11]	S&P 500 + event text	Event-driven deep model	Direction + simulation uplift	Event extraction and alignment complexity

Table 1 highlights two consistent observations. First, reported gains are highly protocol-sensitive (split strategy, feature engineering, horizon definition, and market choice). Second, literature still under-reports risk-adjusted and deployment-relevant outcomes compared with raw accuracy metrics.

2.6 Limitations of Existing Systems

2.6.1 General Modeling Limitations

Across method families, three recurring weaknesses appear: (1) regime instability, where relationships learned in one period degrade in another; (2) over-reliance on point forecasts without calibrated uncertainty; and (3) heavy dependence on handcrafted features or expensive tuning for transfer across assets and markets.

2.6.2 Sentiment Analysis and Integration Gaps

Even when sentiment is included, integration quality remains the bottleneck:

- **Temporal mismatch**: text arrives asynchronously, while price labels are market-clock dependent.
- **Domain drift**: financial language, ticker conventions, and event vocabulary evolve rapidly.
- **Confidence blindness**: many pipelines use polarity scores without reliability weighting.
- **Aggregation mismatch**: document-level polarity may hide aspect-level conflict (e.g., positive revenue, negative guidance).
- **Evaluation mismatch**: many studies optimize RMSE/MAE but under-report trading-risk metrics.

2.7 Comparative Study / Gap Analysis

The literature supports hybrid design, but not a single universally dominant architecture. The unresolved gap is a leakage-safe, sentiment-aware hybrid workflow that simultaneously addresses temporal alignment, domain adaptation, uncertainty handling, and robust out-of-sample evaluation under changing regimes. This directly motivates a methodology that separates linear and non-linear structure, then injects sentiment as an explicitly engineered exogenous signal instead of an opaque black-box add-on.

2.8 Summary of Literature Review

The review establishes five conclusions. First, stock forecasting has evolved from interpretable statistical baselines to ML, deep learning, and hybrid systems. Second, evidence quality depends strongly on protocol quality, not only model class. Third, hybrid models are usually more robust than single-model systems across heterogeneous conditions. Fourth, sentiment can add predictive signal, but only when domain adaptation and time alignment are handled carefully. Fifth, current literature still leaves a practical deployment gap around risk-aware evaluation, uncertainty quantification, and regime adaptation. These gaps define the problem addressed in the next chapter.

3 Problem Identification

3.1 Problem Statement

Despite substantial progress in stock forecasting, reliable next-day prediction remains difficult because financial markets are non-stationary, noisy, and highly sensitive to exogenous information shocks. Price-only pipelines can model historical structure, but they remain reactive to new information and therefore lag event-driven moves.

3.2 Research Gap to be Solved

From Chapter 2, the core unresolved gap is not the absence of powerful models, but the absence of a consistent, leakage-safe integration strategy for combining:

- linear and seasonal price structure,
- non-linear residual behavior,
- and temporally aligned sentiment signals.

Existing work often addresses one or two of these elements, but rarely all three in a reproducible, deployment-oriented pipeline.

3.3 Why Existing Methods Are Insufficient

Current single-model systems underperform during regime shifts or event-heavy periods. Many hybrid systems improve accuracy but still rely on weak sentiment alignment, static weighting, and limited uncertainty reporting. As a result, published gains may not transfer reliably to realistic out-of-sample trading workflows.

3.4 Research Objectives and Hypotheses

Objectives:

1. Build a hybrid forecasting workflow that separates linear trend/seasonal structure from non-linear residual learning.
2. Integrate domain-adapted sentiment features into residual learning using leakage-safe temporal assignment.
3. Evaluate whether sentiment features improve directional and error-based performance relative to a price-only baseline.

Hypotheses:

- H1: Residual-based hybrid modeling outperforms standalone statistical and standalone ML baselines.

- H2: Sentiment-augmented residual features improve directional performance over the price-only hybrid baseline.
- H3: Leakage-safe temporal alignment of news to trading days yields more reliable out-of-sample behavior than naive same-date matching.

3.5 Expected Challenges

- **Data alignment**: matching asynchronous news events to tradable time windows without look-ahead leakage.
- **Regime sensitivity**: maintaining stability when sentiment-price coupling changes across market conditions.
- **Feature reliability**: controlling multicollinearity and preserving interpretability in expanded feature spaces.
- **Evaluation realism**: prioritizing walk-forward, out-of-sample validation over single split optimism.

4 Methodology

This chapter details the implemented forecasting pipeline, from data collection and preprocessing to feature engineering, model development, and evaluation.

4.1 System Architecture Overview

The stock prediction system uses a layered architecture that combines statistical and machine-learning components to model both linear and non-linear behavior in prices. Figure 1 shows the complete data flow from raw market/news inputs to preprocessing, feature engineering, model training, and prediction output.

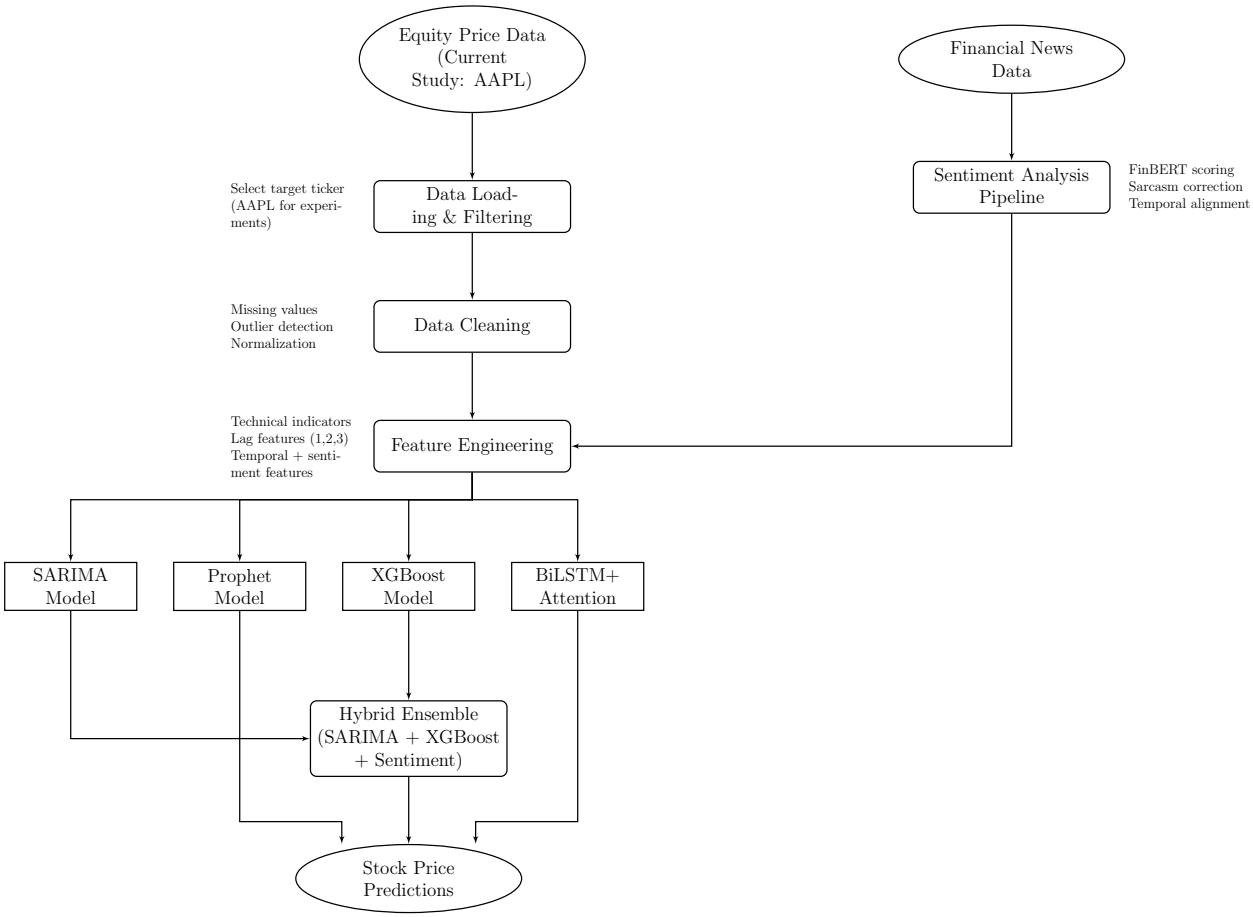

Figure 1: Complete System Architecture for Sentiment-Augmented Hybrid Stock Prediction

The architecture consists of seven primary components:

1. **Data Source**: Historical equity price data from Yahoo Finance and financial news from multiple sources (current experiments use AAPL)

2. **Data Collection & Preprocessing**: Data cleaning, missing value handling, and train-test splitting

3. **Feature Engineering**: Creation of technical indicators, temporal features, and lag variables
4. **Sentiment Analysis Pipeline**: Domain-adapted FinBERT sentiment scoring with dual-head sarcasm correction, temporal alignment, and confidence-weighted aggregation
5. **Model Development**: Implementation of multiple prediction models (SARIMA, Prophet, XGBoost, BiLSTM+Attention)
6. **Hybrid Ensemble**: Combination of SARIMA for trend capture and XGBoost for sentiment-augmented residual learning
7. **Prediction Output**: Price forecasts with model comparison and evaluation metrics

Such a multi-model strategy enables us to use the advantages of each methodology and counterbalance the weakness of the other. The integration of a dedicated sentiment analysis pipeline provides the system with information about exogenous market-moving events that price-only features cannot capture.

4.2 Data Collection and Preprocessing

4.2.1 Data Source and Characteristics

We use the `yfinance` Python library to retrieve historical market data. The framework supports multi-stock experiments (e.g., S&P 500 constituents), but the implemented experiments in this report are single-stock (AAPL). We collect:

- **OHLCV Data**: Open, High, Low, Close prices, and trading Volume
- **Temporal Range**: Multi-year historical data to capture various market conditions
- **Frequency**: Daily stock prices with timestamps

The raw data has the following major attributes:

- `Date`: Trading date (index)
- `Open`: Opening price for the trading day
- `High`: Highest price during the trading day
- `Low`: Lowest price during the trading day
- `Close`: Closing price (primary target variable)
- `Volume`: Number of shares traded
- `Adj Close`: Adjusted closing price accounting for corporate actions

4.2.2 Data Cleaning

The preprocessing pipeline contains three key choices with explicit rationale:

Missing Value Handling: Missing values may arise from market holidays, suspensions, or source-side gaps. We use:

- **Forward Fill**: for isolated gaps where carrying the previous level is a reasonable local approximation.
- **Interpolation**: for short multi-day gaps to avoid discontinuities in derived indicators.
- **Removal**: for series with missingness >5%, where imputation risk outweighs retention benefit.

Outlier Detection and Treatment: Radical price changes may cause model training to be inaccurate. We implement:

$$\text{Outlier} = |x_t - \mu| > 3\sigma \tag{1}$$

where μ represents the rolling mean and σ represents the rolling standard deviation over a 20-day window.

Normalization: For feature-space consistency across indicators with different scales, we apply Min-Max scaling:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{2}$$

4.2.3 News Data Collection

Financial news articles are collected from publicly available sources to serve as input for the sentiment analysis pipeline. Sources include Yahoo Finance News, Reuters financial RSS feeds, and the Financial Modeling Prep news API. For each article, we collect the following data fields:

- **Headline**: Title of the article
- **Body**: Full article text where available; headline-only for paywalled sources
- **Publication Timestamp**: Exact datetime of publication in UTC
- **Source**: Publisher identifier (e.g., Reuters, Bloomberg, Yahoo Finance)
- **Ticker Mention**: Binary indicator of whether the article explicitly mentions the target stock ticker (e.g., AAPL)

The news corpus spans the same temporal range as the price data (2010–2025). Article volume varies considerably over time: high-profile earnings announcements or market-moving events may generate dozens of articles per day, while quiet trading periods may produce none. This variation is explicitly captured through normalized news volume features in the sentiment pipeline. Articles are stored with their original UTC timestamps to enable the principled temporal alignment procedure described in Section 4.3.5.

4.2.4 Train-Test Split

We use a temporal split strategy in order to preserve the sequence of time-series information:

- **Training Set**: First 80% of the chronological data
- **Testing Set**: Final 20% for out-of-sample evaluation
- **Validation**: Time-series cross-validation with expanding window

This will eliminate data leakage and allow realistic evaluation which is a realistic representation of actual trading situations where the future data is unknown.

4.3 Feature Engineering

The so-called feature engineering is where the domain knowledge is coded to the model. The features we implement generate three classes of features that profiled various facets of marketing behavior.

4.3.1 Technical Indicators

Technical analysis indicators are used to measure momentum, trend, volatility and volume. Some of the indicators that we apply are as follows with the help of the Python library of technical analysis:

1. Moving Average Convergence Divergence (MACD):

$$\text{MACD} = \text{EMA}_{12}(\text{Close}) - \text{EMA}_{26}(\text{Close}) \tag{3}$$

$$\text{Signal Line} = \text{EMA}_{9}(\text{MACD}) \tag{4}$$

$$\text{MACD Histogram} = \text{MACD} - \text{Signal Line} \tag{5}$$

MACD shows the direction of the trend and momentum through comparisons of the short-term average and the long-term average which are exponential moving averages.

2. Relative Strength Index (RSI):

$$\text{RSI} = 100 - \frac{100}{1 + \frac{\text{Avg Gain}_{14}}{\text{Avg Loss}_{14}}} \tag{6}$$

RSI is a momentum scale ranging between 0 and 100, where scores of above 70 signal overbought markets and those of below 30 signal oversold markets.

3. Bollinger Bands:

$$\text{Middle Band} = \text{SMA}_{20}(\text{Close}) \tag{7}$$

$$\text{Upper Band} = \text{Middle Band} + 2 \times \sigma_{20} \tag{8}$$

$$\text{Lower Band} = \text{Middle Band} - 2 \times \sigma_{20} \tag{9}$$

Bollinger Bands are used to gauge the price volatility based on the 20 days moving average.

4. Simple Moving Averages (SMA):

$$\text{SMA}_n = \frac{1}{n} \sum_{i=0}^{n-1} \text{Close}_{t-i} \tag{10}$$

We calculate SMAs for multiple windows ($n = 5, 10, 20, 50, 200$) to capture different trend timescales.

5. Exponential Moving Average (EMA):

$$\text{EMA}_t = \alpha \cdot \text{Close}_t + (1 - \alpha) \cdot \text{EMA}_{t-1} \tag{11}$$

$$\text{where } \alpha = \frac{2}{n+1} \tag{12}$$

EMA also places more emphasis on the current prices; hence it is more sensitive to new information than SMA.

Additional Technical Indicators:

- **Stochastic Oscillator**: Measures momentum by comparing closing price to price range
- **Average True Range (ATR)**: Quantifies volatility
- **On-Balance Volume (OBV)**: Cumulative volume indicator for price trend confirmation
- **Commodity Channel Index (CCI)**: Identifies cyclical trends

4.3.2 Temporal Features

There are high calendar effects in market behavior. We derive time characteristics to reflect such patterns:

- **Day of Week**: One-hot encoded (Monday=0, ..., Friday=4)
- **Month**: One-hot encoded (January=1, ..., December=12)
- **Quarter**: Business quarters (Q1, Q2, Q3, Q4)
- **Is Month End**: Binary indicator for month-end trading effects
- **Is Quarter End**: Binary indicator for quarter-end rebalancing

These features enable models to learn the "January effect," "weekend effect," and other calendar-based anomalies documented in financial literature.

4.3.3 Lag Features

Time-series prediction depends on short- and medium-memory effects. Lag features are defined on the closing price:

$$\text{Lag}_k = \text{Close}_{t-k} \quad \text{for } k \in \{1, 2, 3, 5, 10\} \tag{13}$$

These lag choices balance short-horizon reaction effects (1–3 days) and slightly longer carryover structure (5, 10 days), while keeping dimensionality manageable for tree-based learners.

4.3.4 Feature Selection and Dimensionality

Once we have feature engineered, we have about 25–30 price-derived features in our data. With the addition of 8 sentiment features (described in Section 4.3.6), the total feature space expands to approximately $d_p + 8 \approx 33$–38 dimensions. In order to avoid multicollinearity and overfitting:

- **Correlation Analysis**: Remove features with pairwise correlation > 0.95
- **Feature Importance**: Use XGBoost feature importance (ranked by gain) to identify top predictive features
- **Domain Filtering**: Retain features with established financial theory support
- **Variance Inflation Factor (VIF)**: Remove features with VIF > 10 to control multicollinearity in the expanded feature space

4.3.5 Sentiment Analysis Pipeline

The sentiment analysis pipeline transforms raw financial news articles into structured, quantitative sentiment signals suitable for integration into the prediction model. The pipeline addresses the critical gaps identified in the literature review—sarcasm detection, tokenization drift, domain sensitivity, temporal alignment, and uncertainty quantification—through a four-stage process. Figure 2 illustrates the complete end-to-end data flow from raw news articles to the final sentiment feature vector.

Base Model: FinBERT

The sentiment scoring component is built on FinBERT, a domain-adapted variant of BERT (Bidirectional Encoder Representations from Transformers). BERT is a transformer-based neural network that uses multi-head self-attention to learn contextualized word representations. Unlike static word embeddings that assign a fixed vector to each word regardless of context, BERT produces context-dependent representations: the word "crash" receives a different embedding in "stock market crash" than in "application crash."

FinBERT inherits BERT's 12-layer, 768-dimensional transformer encoder architecture (approximately 110 million parameters) and is further pre-trained on a large corpus of financial communications including financial news articles, earnings call transcripts, and analyst reports. This domain-specific pre-training allows FinBERT to correctly interpret financial expressions

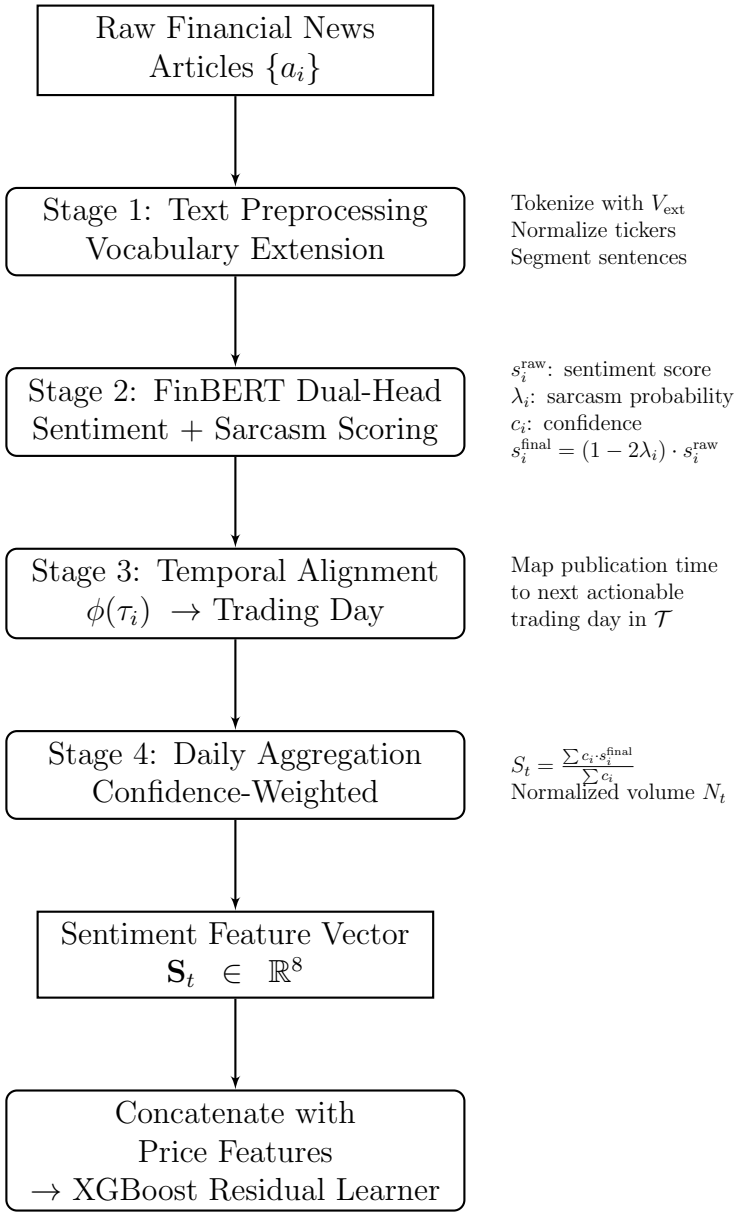

Figure 2: Sentiment Analysis Pipeline: End-to-End Data Flow from Raw News to XGBoost Integration

that general-purpose BERT misclassifies—for example, FinBERT understands that "the company beat earnings expectations" carries positive sentiment, whereas general BERT may be uncertain about the polarity of "beat" in a financial context.

The sentiment classification head maps the `[CLS]` token embedding—a 768-dimensional vector that summarizes the entire input sequence—to a three-class probability distribution (Positive, Negative, Neutral) via a fully connected layer and softmax activation. In our dual-head architecture, a second fully connected layer branches from the same `[CLS]` embedding to produce a binary sarcasm probability via sigmoid activation. Both heads share the same transformer backbone, so financial language understanding learned by the sentiment head also benefits sarcasm detection.

Training Data and Fine-Tuning Procedure

The dual-head model is fine-tuned on two labeled datasets:

1. **Sentiment Labels**: The Financial Phrasebank dataset provides approximately 4,840 English financial news sentences labeled by 16 financial experts as Positive, Negative, or Neutral. We use the subset with $\geq 75\%$ annotator agreement to ensure label quality.

2. **Sarcasm Labels**: A curated financial sarcasm dataset containing approximately 3,000 labeled examples, constructed by collecting financial headlines with confirmed sarcastic intent (marked by expert annotators) and augmenting with non-sarcastic financial headlines as negative examples.

Fine-tuning proceeds in two phases to prevent catastrophic forgetting of the pre-trained representations:

1. **Phase 1 — Frozen Backbone**: The FinBERT transformer weights are frozen. Only the two classification heads (sentiment and sarcasm) are trained for 5 epochs with a learning rate of 2×10^{-4}. This allows the heads to initialize without corrupting the pre-trained language representations.

2. **Phase 2 — Joint Fine-Tuning**: All model weights (backbone + both heads) are unfrozen, and the entire model is trained end-to-end for 3 additional epochs with a reduced learning rate of 2×10^{-5}. The combined training objective is:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{sentiment}} + (1 - \alpha) \cdot \mathcal{L}_{\text{sarcasm}} \tag{14}$$

   where $\alpha = 0.7$ prioritizes the primary sentiment task, $\mathcal{L}_{\text{sentiment}}$ is categorical cross-entropy over three classes, and $\mathcal{L}_{\text{sarcasm}}$ is binary cross-entropy. Training uses the AdamW optimizer with a batch size of 32 and linear learning rate warmup over the first 10% of training steps.

The four processing stages of the pipeline are described below.

Stage 1: Text Preprocessing with Vocabulary Extension

Before sentiment scoring, each article undergoes domain-aware preprocessing. The standard FinBERT tokenizer is extended with a custom financial vocabulary containing common ticker

symbols, financial ratios, and regulatory terms to address the tokenization drift gap. Given the original FinBERT vocabulary V_{FinBERT}, the extended vocabulary is:

$$V_{\text{ext}} = V_{\text{FinBERT}} \cup V_{\text{fin}} \tag{15}$$

where V_{fin} is a curated set of financial domain tokens including ticker symbols, financial ratios (P/E, EV/EBITDA), and regulatory terminology (Form 10-K, material adverse change). Ticker symbols in the format \$TICKER are normalized to a canonical form TICKER_REF to prevent subword fragmentation. URLs, HTML entities, and boilerplate disclosure text are removed, while numerical values are retained as they carry information about financial magnitudes.

Long articles are segmented into individual sentences, and each sentence is scored independently before aggregation at the document level. This preserves local sentiment nuance that document-level scoring would otherwise lose.

Stage 2: Dual-Head Sentiment Scoring

The core of the pipeline is a dual-head classification model built on the FinBERT transformer backbone. The model has two output heads sharing the same encoder:

- **Head 1 (Sentiment Head)**: A three-class classifier (Positive, Negative, Neutral) producing a probability distribution $P_{\text{sent}}(c \mid a_i)$ for article a_i.
- **Head 2 (Sarcasm Head)**: A binary classifier producing the probability of sarcasm $P_{\text{sarc}}(a_i)$.

The raw sentiment score is converted to a continuous scale:

$$s_i^{\text{raw}} = P_{\text{sent}}(\text{Positive} \mid a_i) - P_{\text{sent}}(\text{Negative} \mid a_i) \in [-1, 1] \tag{16}$$

The sarcasm-corrected sentiment score inverts the raw score in proportion to the estimated probability of sarcasm:

$$s_i^{\text{final}} = (1 - 2\lambda_i) \cdot s_i^{\text{raw}} \tag{17}$$

where $\lambda_i = P_{\text{sarc}}(a_i) \in [0, 1]$. When $\lambda_i = 0$ (no sarcasm), the raw score is preserved; when $\lambda_i = 1$ (definite sarcasm), the sentiment polarity is fully inverted. The confidence of the prediction is defined as:

$$c_i = \max_{k \in \{\text{Pos,Neg,Neu}\}} P_{\text{sent}}(k \mid a_i) \in \left[\tfrac{1}{3}, 1\right] \tag{18}$$

Stage 3: Temporal Alignment

Let τ_i denote the UTC publication timestamp of article a_i. The trading-day assignment function $\phi(\tau_i)$ maps each article to the appropriate trading day:

$$\phi(\tau_i) = \min\{t \in \mathcal{T} : \text{MarketOpen}(t) > \tau_i\} \tag{19}$$

where $\mathcal{T}$ is the set of valid NYSE trading days. This formal assignment ensures:

- Articles published during or after market hours on day t are assigned to the next trading day $t + 1$.

- Articles published after market close on Friday are assigned to Monday (or the next valid trading day if Monday is a holiday).
- No future information leaks into the training set.

Stage 4: Daily Sentiment Aggregation

For trading day t, let $\mathcal{A}_t = \{a_i : \phi(\tau_i) = t\}$ be the set of articles assigned to that day. The confidence-weighted daily sentiment score is:

$$S_t = \frac{\sum_{a_i \in \mathcal{A}_t} c_i \cdot s_i^{\text{final}}}{\sum_{a_i \in \mathcal{A}_t} c_i} \tag{20}$$

If $\mathcal{A}_t = \emptyset$ (no news on trading day t), then $S_t = 0$ (neutral). The daily news volume, normalized over the dataset, is:

$$N_t = \frac{|\mathcal{A}_t| - \mu_N}{\sigma_N} \tag{21}$$

where μ_N and σ_N are the mean and standard deviation of daily article counts over the training period.

End-to-End Daily Processing Loop

The complete pipeline operates as a daily batch process executed after market close. For each trading day t, the system performs the following sequence:

1. **Collect**: Retrieve all news articles $\{a_i\}$ published since the last processing run from the configured news sources (Yahoo Finance, Reuters RSS, Financial Modeling Prep API).
2. **Preprocess**: For each article a_i, apply vocabulary-extended tokenization, ticker normalization, boilerplate removal, and sentence segmentation (Stage 1).
3. **Score**: Pass each preprocessed sentence through the dual-head FinBERT model. The sentiment head produces the raw score s_i^{raw} and confidence c_i; the sarcasm head produces λ_i. Compute the sarcasm-corrected score $s_i^{\text{final}} = (1 - 2\lambda_i) \cdot s_i^{\text{raw}}$ (Stage 2). For multi-sentence articles, aggregate sentence-level scores to a single document-level score weighted by sentence confidence.
4. **Align**: Apply the temporal assignment function $\phi(\tau_i)$ to each article's UTC timestamp to determine the correct target trading day. Articles published after today's market close are assigned to the next trading day (Stage 3).
5. **Aggregate**: For the target trading day t, compute the confidence-weighted daily sentiment S_t and the normalized news volume N_t from all articles in $\mathcal{A}_t$. If $\mathcal{A}_t = \emptyset$, set $S_t = 0$ and flag $I_t^{\text{no\_news}} = 1$ (Stage 4).
6. **Derive Features**: Using the aggregated S_t and the historical sentiment values $S_{t-1}, S_{t-2}, S_{t-3}$, compute the full 8-dimensional sentiment feature vector $\mathbf{S}_t = [S_t, \Delta S_t, \sigma_{S,t}, N_t, D_t, S_{t-1}, S_{t-2}, S_{t-3}]$.

7. **Integrate**: Concatenate $\mathbf{S}_t$ with the price-derived feature vector $\mathbf{X}_t^{\text{price}}$ to form $\mathbf{X}_t^{\text{ext}}$, which is passed to the XGBoost residual learner alongside the SARIMA baseline prediction.

This daily loop ensures strict temporal causality: sentiment features for trading day t are computed exclusively from articles published before day t's market open, preventing any form of information leakage. The entire processing cycle (collection, scoring, feature computation) takes approximately 2–5 minutes on standard CPU hardware depending on news volume, making it feasible as an overnight batch process.

4.3.6 Sentiment Feature Engineering

From the raw daily sentiment signal S_t and the news volume N_t produced by the sentiment pipeline, we derive a structured set of eight features that capture different aspects of the sentiment dynamics.

1. Daily Aggregated Sentiment (S_t): The confidence-weighted, sarcasm-corrected daily sentiment score as defined in Section 4.3.5.

2. Sentiment Momentum (ΔS_t):

$$\Delta S_t = S_t - S_{t-1} \tag{22}$$

Captures sudden shifts in news sentiment that may precede price movements.

3. Rolling Sentiment Volatility ($\sigma_{S,t}$):

$$\sigma_{S,t} = \sqrt{\frac{1}{k}\sum_{j=0}^{k-1}\left(S_{t-j} - \bar{S}_{t,k}\right)^2} \tag{23}$$

where $k = 5$ (one trading week) and $\bar{S}_{t,k} = \frac{1}{k}\sum_{j=0}^{k-1} S_{t-j}$. High sentiment volatility indicates an unstable news environment and may signal increased price volatility.

4. Normalized News Volume (N_t): Z-score normalized daily article count, capturing the intensity of news coverage. Abnormally high news volume often accompanies significant price-moving events.

5. Sentiment-Price Divergence (D_t):

$$D_t = S_{t-1} - \text{sign}\left(\text{Close}_{t-1} - \text{Close}_{t-2}\right) \tag{24}$$

Measures the alignment between the prior day's sentiment and price movement. A large positive divergence (positive sentiment but negative return) may signal a mean-reversion opportunity.

6–8. Lagged Sentiment Features ($S_{t-1}, S_{t-2}, S_{t-3}$):

$$S_{t-k} \quad \text{for } k \in \{1, 2, 3\} \tag{25}$$

Capture the delayed reaction of prices to news sentiment over a three-day horizon.

The full sentiment feature vector for day t is:

$$\mathbf{S}_t = [S_t,\ \Delta S_t,\ \sigma_{S,t},\ N_t,\ D_t,\ S_{t-1},\ S_{t-2},\ S_{t-3}] \in \mathbb{R}^8 \tag{26}$$

This structured representation provides the downstream XGBoost model with richer and more reliable sentiment signals than raw point estimates, addressing the uncertainty quantification gap identified in the literature review.

4.4 Model Development

There are five types of prediction models that we apply and each of them reflects various features of the stock prices dynamics.

4.4.1 SARIMA Model

Model Overview: Seasonal AutoRegressive Integrated Moving Average (SARIMA) extends ARIMA by explicitly modeling seasonal patterns. The model is specified as SARIMA$(p, d, q) \times (P, D, Q)_s$ where:

- (p, d, q): Non-seasonal AR order, differencing, MA order
- (P, D, Q): Seasonal AR order, differencing, MA order
- s: Seasonality period (e.g., 5 for weekly patterns in daily data)

Mathematical Formulation:

$$\phi_p(B)\Phi_P(B^s)(1-B)^d(1-B^s)^D y_t = \theta_q(B)\Theta_Q(B^s)\epsilon_t \tag{27}$$

where:

- B: Backward shift operator ($B^k y_t = y_{t-k}$)
- $\phi_p(B)$: Non-seasonal AR polynomial
- $\Phi_P(B^s)$: Seasonal AR polynomial
- $\theta_q(B)$: Non-seasonal MA polynomial
- $\Theta_Q(B^s)$: Seasonal MA polynomial
- ϵ_t: White noise error term

Implementation Details: We use the `pmdarima` library's `auto_arima` function for automated parameter selection:

- **Order Selection**: Auto-ARIMA searches over parameter space to minimize AIC/BIC
- **Stationarity Testing**: Augmented Dickey-Fuller test for unit roots

- **Seasonality Detection**: Automatic identification of seasonal patterns
- **Final Configuration**: SARIMA$(0,1,0) \times (0,0,0)_{[5]}$ (identified by auto-fitting)

Training Process:

1. Test for stationarity using ADF test
2. Apply differencing if necessary to achieve stationarity
3. Fit SARIMA model using maximum likelihood estimation
4. Validate residuals for white noise properties (Ljung-Box test)
5. Generate forecasts for test period

4.4.2 Prophet Model

Model Overview: Prophet, developed by Facebook (Meta), employs an additive model framework particularly suited for business time-series with strong seasonal effects and multiple trend changes.

Mathematical Formulation:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \tag{28}$$

where:

- $g(t)$: Piecewise linear or logistic growth trend
- $s(t)$: Periodic component (daily, weekly, yearly seasonality)
- $h(t)$: Holiday/event effects
- ϵ_t: Idiosyncratic error term

Trend Component: Prophet determines non-linear trends with the help of changepoint:

$$g(t) = (k + \mathbf{a}(t)^T\delta)t + (m + \mathbf{a}(t)^T\gamma) \tag{29}$$

where the value of delta is rate changes at changepoints that are automatically identified.
Seasonality Component: Fourier series is used to model seasonality:

$$s(t) = \sum_{n=1}^{N} \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right) \tag{30}$$

where P is the period (365.25 for yearly, 7 for weekly).
Implementation Details:

- **Library**: `fbprophet` Python package

- **Changepoint Prior**: 0.05 (controls flexibility of trend)
- **Seasonality Mode**: Multiplicative (appropriate for percentage changes)
- **Weekly Seasonality**: Enabled to capture weekday effects
- **Uncertainty Intervals**: 95% prediction intervals via MCMC sampling

4.4.3 XGBoost Model

Model Overview: XGBoost (eXtreme Gradient Boosting) is a system that adopts optimized gradient boosting decision trees. Compared to other statistical models, XGBoost is able to automatically formulate non-linear interactions between features and can work effectively with mixed data types.

Mathematical Formulation: XGBoost creates an additive regression of the weak decision trees learners:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F} \tag{31}$$

where $\mathcal{F}$ represents the space of regression trees, and each tree f_k is learned to minimize the objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \tag{32}$$

The overfitting is avoided by the regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda ||\mathbf{w}||^2 \tag{33}$$

where T is the number of leaves and $\mathbf{w}$ represents leaf weights.

Implementation Details:

- **Library**: `xgboost` Python package
- **Objective Function**: `reg:squarederror` (regression)
- **Number of Estimators**: 100 trees
- **Learning Rate**: 0.1 (step size shrinkage)
- **Max Depth**: 5 (maximum tree depth)
- **Subsample**: 0.8 (row sampling ratio)
- **Colsample by Tree**: 0.8 (column sampling ratio)
- **Early Stopping**: Monitor validation RMSE with patience=10

Feature Set: XGBoost makes use of the entire engineered set that consists of:

- All technical indicators (MACD, RSI, Bollinger Bands, etc.)

- Temporal features (day of week, month, quarter)
- Lag features (1, 2, 3, 5, 10 days)
- Volume-derived features

Hyperparameter Optimization: We use grid search and time-series cross-validation to optimize hyperparameters in the search space:

- Learning rate: $\{0.01, 0.05, 0.1, 0.2\}$
- Max depth: $\{3, 5, 7, 10\}$
- Number of estimators: $\{50, 100, 200, 300\}$

4.4.4 BiLSTM + Attention Model

Model Overview: The Bidirectional Long Short-Term Memory (BiLSTM) networks are coupled with a modification of the attention process in our deep learning architecture. The architecture offers better results than vanilla LSTMs because it takes into account the constraints of processing sequences in both time directions and selectively concentrating on the most informative time steps.

Architecture Components:

1. Input Layer:

- **Shape**: (batch_size, timesteps, features)
- **Configuration**: $(None, 10, 4)$ — 10 historical days, 4 features (Open, High, Low, Close)

2. Bidirectional LSTM Layer:

The BiLSTM works off of the input sequence in both directions:

$$\overrightarrow{h}_t = \text{LSTM}_{\text{forward}}(x_t, \overrightarrow{h}_{t-1}) \tag{34}$$

$$\overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(x_t, \overleftarrow{h}_{t+1}) \tag{35}$$

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \tag{36}$$

Both LSTM cells have gating mechanisms:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(Forget gate)} \tag{37}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(Input gate)} \tag{38}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{(Candidate values)} \tag{39}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(Cell state)} \tag{40}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(Output gate)} \tag{41}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(Hidden state)} \tag{42}$$

- **Units**: 100 (50 forward + 50 backward)

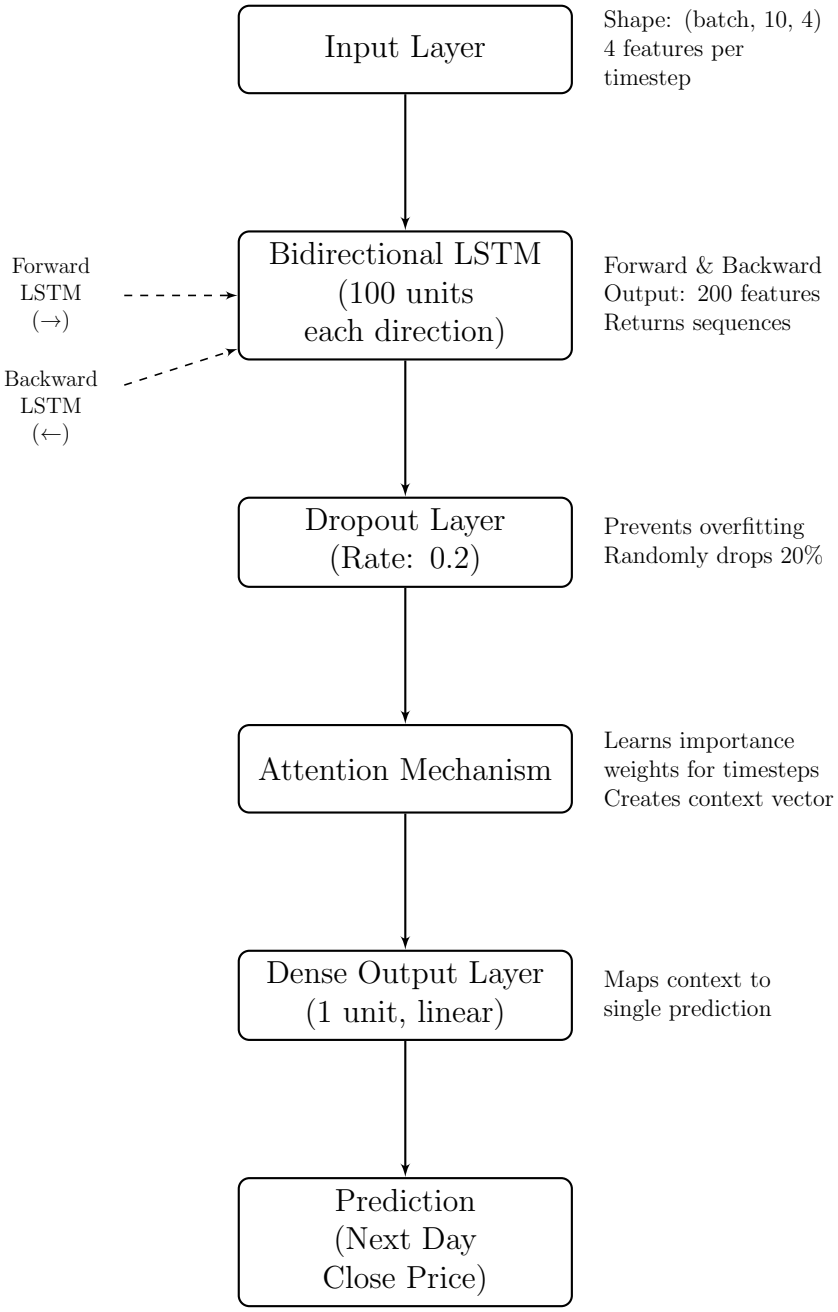

Figure 3: BiLSTM with Attention Mechanism Architecture

- **Return Sequences**: True (outputs sequence for attention layer)
- **Activation**: `tanh` (cell state), `sigmoid` (gates)

3. Dropout Layer:

- **Dropout Rate**: 0.2
- **Purpose**: Prevent overfitting by randomly zeroing 20% of outputs

4. Custom Attention Mechanism:
Our layer of attention is trained to give the weight of various time steps:

$$e_t = \tanh(W_a \cdot h_t + b_a) \quad \text{(Attention scores)} \tag{43}$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{i=1}^{T} \exp(e_i)} \quad \text{(Normalized weights)} \tag{44}$$

$$c = \sum_{t=1}^{T} \alpha_t h_t \quad \text{(Context vector)} \tag{45}$$

where:

- W_a, b_a: Trainable attention parameters
- α_t: Attention weights (sum to 1)
- c: Weighted sum of hidden states (context vector)

The attention mechanism helps the model to concentrate on important patterns (e.g. recent volatility spikes, trend reversals) and on less informative periods gives less weight.

5. Dense Output Layer:

- **Units**: 1 (scalar price prediction)
- **Activation**: Linear (for regression)

Model Compilation:

- **Optimizer**: Adam with learning rate = 0.001
- **Loss Function**: Mean Squared Error (MSE)
- **Metrics**: MAE, RMSE

Training Configuration:

- **Epochs**: 50 with early stopping (patience=10)
- **Batch Size**: 32

- **Validation Split**: 20% of training data for validation
- **Callbacks**: ModelCheckpoint (save best model), EarlyStopping, ReduceLROnPlateau

Data Preparation for BiLSTM: We take sliding windows on the time series:

- **Sequence Length**: 10 days
- **Prediction Horizon**: 1 day ahead (next closing price)
- **Stride**: 1 day (overlapping windows)

For a dataset with N days, we generate $N - 10$ training samples, each containing 10 consecutive days of features and the following day's close price as the target.

4.4.5 Hybrid SARIMA-XGBoost Model

Model Overview: The hybrid model incorporates the complementary power of SARIMA and XGBoost with the help of a two-stage residual learning structure. SARIMA takes in linear trends and seasonality, and XGBoost trains the non-linear residual trends which SARIMA is unable to capture.

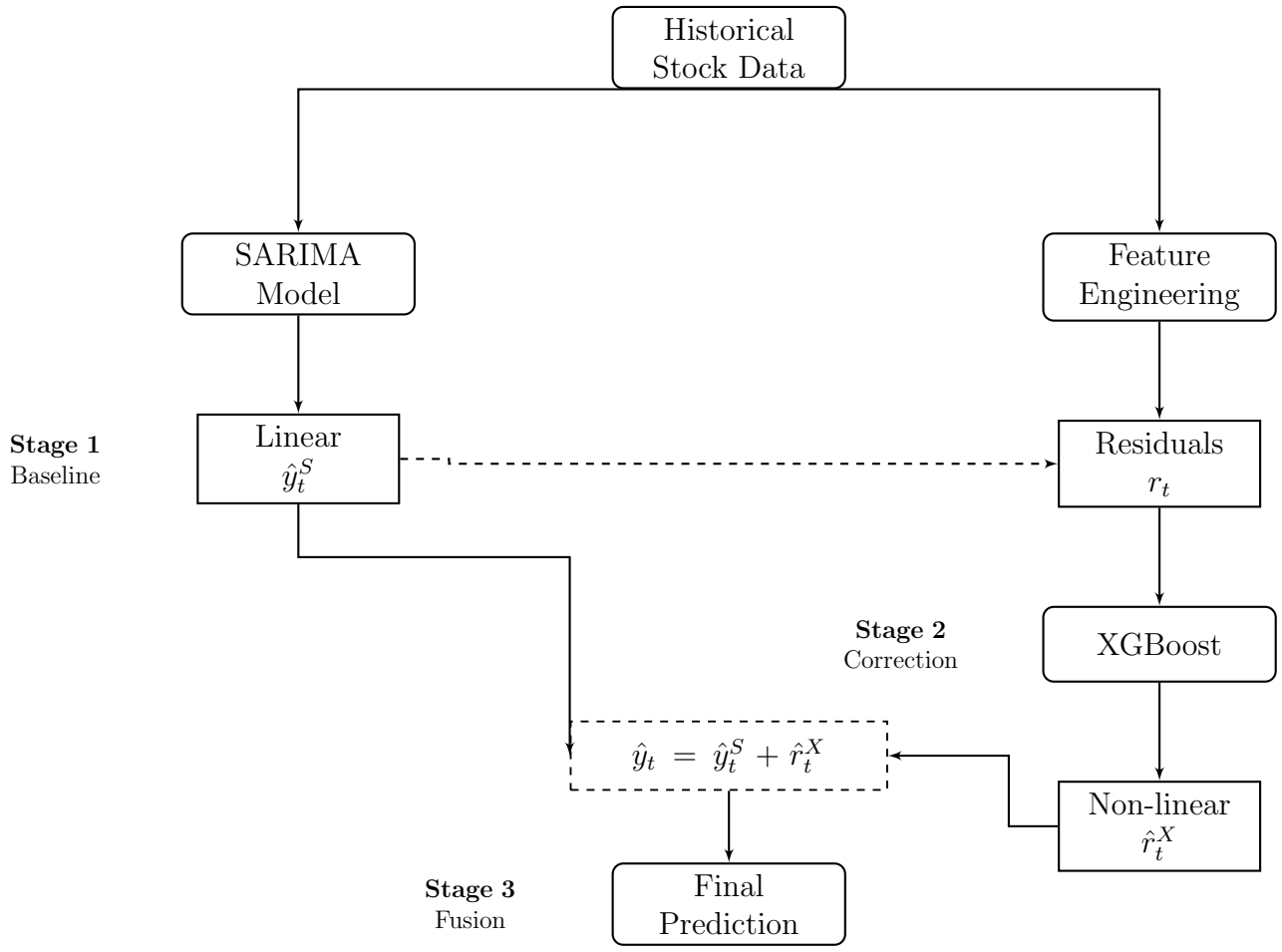

Figure 4: Hybrid SARIMA-XGBoost Workflow

Hybrid Architecture Rationale:
Financial time series are a two sided phenomenon:

$$y_t = L_t + N_t + \epsilon_t \tag{46}$$

where:

- L_t: Linear component (trends, seasonality, autocorrelation)
- N_t: Non-linear component (regime shifts, volatility clustering, feature interactions)
- ϵ_t: Random noise

SARIMA excels at modeling L_t through its ARMA structure, while XGBoost captures N_t through its decision tree ensemble.

Implementation Pipeline:

Stage 1: SARIMA Baseline

1. Train SARIMA$(0,1,0) \times (0,0,0)_{[5]}$ on training data
2. Generate forecasts: $\hat{y}_t^{\text{SARIMA}}$
3. Calculate residuals: $r_t = y_t - \hat{y}_t^{\text{SARIMA}}$

Stage 2: XGBoost Residual Modeling

1. Engineer features from original data (lag features, technical indicators)
2. Train XGBoost to predict residuals: $r_t \sim f(\mathbf{X}_t)$
3. Generate residual predictions: $\hat{r}_t^{\text{XGBoost}}$

Stage 3: Hybrid Fusion

$$\hat{y}_t^{\text{Hybrid}} = \hat{y}_t^{\text{SARIMA}} + \hat{r}_t^{\text{XGBoost}} \tag{47}$$

Mathematical Justification:
Assuming that SARIMA is able to capture the linear component, the residual of SARIMA will only have the non-linear component and noise:

$$r_t = y_t - \hat{y}_t^{\text{SARIMA}} \approx N_t + \epsilon_t \tag{48}$$

XGBoost then approximates:

$$\hat{r}_t^{\text{XGBoost}} \approx N_t \tag{49}$$

The hybrid prediction is the last one, it is:

$$\hat{y}_t^{\text{Hybrid}} \approx L_t + N_t \tag{50}$$

assuming $\hat{y}_t^{\text{SARIMA}} \approx L_t$.

Error Decomposition:

The overall error in prediction may be broken down:

$$\begin{aligned} \text{Error}^{\text{Hybrid}} &= y_t - \hat{y}_t^{\text{Hybrid}} && (51)\\ &= (L_t + N_t + \epsilon_t) - (\hat{L}_t + \hat{N}_t) && (52)\\ &= (L_t - \hat{L}_t) + (N_t - \hat{N}_t) + \epsilon_t && (53) \end{aligned}$$

Through specialization of each component we reduce the linear and non-linear errors separately resulting in reduced overall error compared to single-model methods.

Implementation Advantages:

- **Robustness**: SARIMA provides stable baseline; XGBoost adds adaptive corrections
- **Interpretability**: Decomposition reveals linear vs. non-linear contributions
- **Complementarity**: Each model addresses the other's weaknesses
- **Generalization**: Reduces overfitting risk compared to single complex model

Sentiment-Augmented Extension:

The baseline hybrid decomposes financial time series into linear and non-linear components. However, this two-component formulation is structurally blind to exogenous information shocks—such as earnings surprises, macroeconomic announcements, and shifts in public sentiment—that drive price movements independently of historical patterns. We extend the decomposition to a three-component model:

$$y_t = L_t + N_t + E_t + \epsilon_t \tag{54}$$

where E_t represents the sentiment-driven component of price dynamics that is not captured by either the linear trend (L_t) or the non-linear residual patterns (N_t).

Extended Feature Vector:

The XGBoost residual learner in Stage 2 now operates on an extended feature vector that concatenates price-derived and sentiment features:

$$\mathbf{X}_t^{\text{ext}} = \left[\mathbf{X}_t^{\text{price}},\ \mathbf{S}_t\right] \in \mathbb{R}^{d_p+8} \tag{55}$$

where $\mathbf{X}_t^{\text{price}} \in \mathbb{R}^{d_p}$ contains all price-derived features (lag features, technical indicators, temporal features) with $d_p \approx 25$–30, and $\mathbf{S}_t \in \mathbb{R}^8$ is the structured sentiment feature vector defined in Section 4.3.6.

Sentiment-Augmented Residual Modeling:

XGBoost learns to predict the SARIMA residual using the extended feature vector:

$$\hat{r}_t = f\left(\mathbf{X}_t^{\text{ext}}\right) = \sum_{k=1}^{K} f_k\left(\mathbf{X}_t^{\text{ext}}\right), \quad f_k \in \mathcal{F} \tag{56}$$

The sentiment-augmented hybrid prediction is:

$$\hat{y}_t^{\text{Hybrid+Sent}} = \hat{y}_t^{\text{SARIMA}} + f\left(\mathbf{X}_t^{\text{ext}}\right) \tag{57}$$

Under the three-component decomposition, SARIMA captures L_t and XGBoost (now with sentiment features) captures both N_t and E_t:

$$\hat{y}_t^{\text{Hybrid+Sent}} \approx L_t + N_t + E_t \tag{58}$$

The total prediction error decomposes as:

$$\begin{aligned} \text{Error}_t &= y_t - \hat{y}_t^{\text{Hybrid+Sent}} \\ &= (L_t - \hat{L}_t) + (N_t + E_t - \hat{N}_t - \hat{E}_t) + \epsilon_t \end{aligned} \tag{59}$$

By providing XGBoost with sentiment features, we reduce the $(E_t - \hat{E}_t)$ term—the portion of the residual attributable to sentiment-driven price movements—leading to lower overall prediction error compared to the baseline hybrid.

Sarcasm Detector Training:

The dual-head model's complete training procedure—including datasets, the two-phase fine-tuning schedule, and the combined loss function—is detailed in Section 4.3.5. In summary, the model is fine-tuned first with a frozen FinBERT backbone to initialize the classification heads, then end-to-end with the joint objective $\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{sentiment}} + (1 - \alpha) \cdot \mathcal{L}_{\text{sarcasm}}$ where $\alpha = 0.7$. The sentiment head is supervised by the Financial Phrasebank ($\approx$4,840 expert-labeled sentences) and the sarcasm head by a curated financial sarcasm corpus ($\approx$3,000 labeled examples).

Handling Missing Sentiment Days:

On trading days with no news articles ($\mathcal{A}_t = \emptyset$), S_t is set to 0 (neutral) and N_t is set to the normalized value corresponding to zero articles. A binary indicator feature $I_t^{\text{no\_news}} \in \{0, 1\}$ is added to the feature vector to allow XGBoost to distinguish genuinely neutral sentiment from absent sentiment:

$$\mathbf{X}_t^{\text{ext}} = \left[\mathbf{X}_t^{\text{price}},\ \mathbf{S}_t,\ I_t^{\text{no\_news}}\right] \tag{60}$$

4.5 Evaluation Metrics

We use several evaluation measures to fully evaluate the performance of the models and to measure the various components of prediction quality.

4.5.1 Regression Metrics

1. Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{61}$$

MAE directly translates in the same units as the variable of interest (dollars), and it is therefore intuitive.

2. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{62}$$

The squaring operation of RMSE punishes large errors more than MAE thus being sensitive to outliers.

3. Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{63}$$

MAPE has scale-free during measurement of errors, which means that it can be compared across stocks of varying price levels.

4. R-squared (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{64}$$

R^2 measures the percentage of variance that a model has explained, the closer it is to 1, the more it has been explained.

4.5.2 Directional Accuracy

Direction Accuracy:

$$\text{DA} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}_{\text{sign}(y_i - y_{i-1}) = \text{sign}(\hat{y}_i - y_{i-1})} \tag{65}$$

DA is used to assess the frequency of the model giving correct predictions of the price direction (up/down) which can often be very useful in trading as compared to absolute price accuracy.

4.5.3 Trading Performance Metrics

Sharpe Ratio:

$$\text{SR} = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p} \tag{66}$$

where R_p is the portfolio (strategy) return, R_f is the risk-free rate, and σ_p is the standard deviation of excess returns. The Sharpe Ratio measures the risk-adjusted return of a trading strategy derived from the model's predictions. A higher Sharpe Ratio indicates better risk-adjusted performance, bridging the gap between statistical prediction accuracy and practical trading viability.

4.5.4 Model Comparison

We evaluate each of the five models (SARIMA, Prophet, XGBoost, BiLSTM+Attention, Hybrid) in terms of the following metrics to determine:

- **Best Overall Performer**: Model with lowest RMSE/MAE
- **Most Stable**: Model with lowest variance across cross-validation folds
- **Best Directional**: Model with highest direction accuracy
- **Computational Efficiency**: Training time and prediction latency

4.6 Implementation Details

4.6.1 Development Environment

The system was implemented using Python 3.8+ with core libraries including `pandas` and `numpy` for data manipulation, `scikit-learn` and `xgboost` for machine learning, `tensorflow/keras` for deep learning, and `statsmodels/pmdarima` for statistical forecasting.

4.6.2 Computational Resources

Training of models was done on a conventional processor-based (Intel core i7 equivalent) environment. The time to train statistical models (SARIMA, Prophet) took between 1-3 minutes, whereas the deep learning models (BiLSTM+Attention) took between 10-20 minutes.

4.6.3 Reproducibility

In order to be reproducible, we have used fixed random seeds of all stochastic operations, versioned data snapshots, and a specified dependency environment.

4.7 Summary

The chapter provided a complete approach to predicting hybrid stock prices which included:

1. **System Architecture**: Multi-model pipeline from data collection to prediction, incorporating both price and news data inputs
2. **Data Processing**: Collection of OHLCV price data and financial news articles, cleaning, and train-test splitting strategies
3. **Feature Engineering**: Technical indicators, temporal features, lag variables, and structured sentiment features
4. **Sentiment Analysis Pipeline**: Domain-adapted FinBERT sentiment scoring with dual-head sarcasm correction, principled temporal alignment, confidence-weighted aggregation, and an 8-dimensional structured sentiment feature vector

5. **Model Development**: Five complementary approaches (SARIMA, Prophet, XGBoost, BiLSTM+Attention, Hybrid) with the hybrid model extended to incorporate sentiment features through a three-component decomposition $y_t = L_t + N_t + E_t + \epsilon_t$

6. **Evaluation**: Comprehensive metrics for regression accuracy, directional accuracy, and risk-adjusted trading performance (Sharpe Ratio)

7. **Implementation**: Practical details for reproducibility

The approach is an amalgamation of theoretical and practical application of the strengths of statistical paradigm, machine learning, deep learning, and natural language processing. The three-component decomposition of financial time series is specifically resolved in the sentiment-augmented hybrid architecture, which breaks down predictions into linear trends (L_t), non-linear residual patterns (N_t), and sentiment-driven dynamics (E_t), each addressed by specialized components.

5 Results and Analysis

5.1 Experimental Setup

Experiments were conducted using Apple Inc. (AAPL) historical stock data from 2010 to 2025, comprising approximately 3,773 trading days with OHLCV (Open, High, Low, Close, Volume) features.

Data Split:

- Training set: 80% (3,018 days) - chronologically first portion
- Testing set: 20% (755 days) - chronologically last portion
- No random shuffling to preserve temporal order and prevent data leakage

Evaluation Metrics:

- RMSE: Root Mean Squared Error (penalizes large errors, same units as target)
- R^2: Proportion of variance explained (1.0 = perfect, 0 = mean baseline, ¡0 = worse than mean)
- MAE: Mean Absolute Error (robust to outliers)
- Directional Accuracy: % correct up/down predictions (critical for trading)

5.2 Exploratory Data Analysis

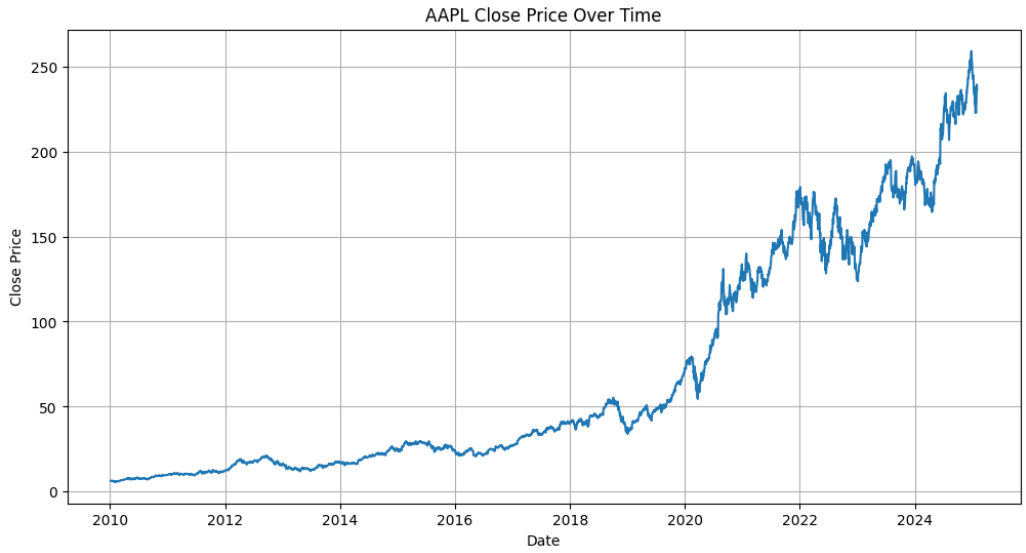

Figure 5: Historical Closing Price of AAPL (2010-2025)

Figure 5 reveals: (1) strong non-stationarity with price increasing from $30 (2010) to ¿$200 (2025), (2) accelerating upward trend post-2019, (3) increasing volatility in recent years, (4)

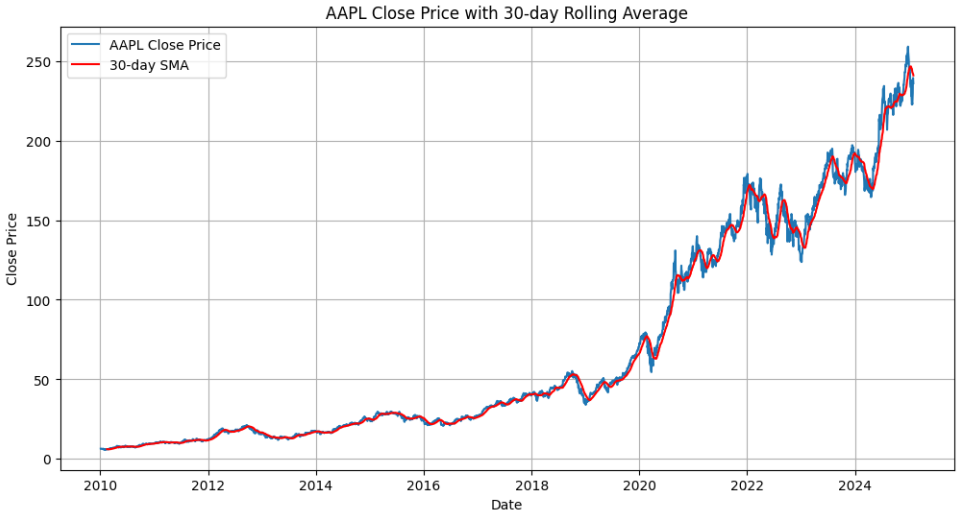

Figure 6: AAPL Close Price with 30-day Rolling Average

clear volatility clustering periods. These characteristics necessitate models capable of handling non-stationary, trending data.

Figure 6 highlights long-run trend behavior with local deviations around the 30-day average, supporting the need for models that capture both baseline structure and short-term residual dynamics. Separate correlation checks on engineered OHLC-derived features still indicated high collinearity among raw price levels, motivating lag/indicator-based feature expansion and feature-selection controls.

5.3 Model Performance Evaluation

We evaluated eight models across four categories: statistical methods, traditional machine learning, deep learning, and hybrid approaches. Table 3 summarizes all results.

5.3.1 Statistical Models

ARIMA failed catastrophically (RMSE: 32.52, R^2: -0.0868). The negative R^2 indicates performance worse than predicting the mean. Failure reasons: (1) inadequate trend handling despite differencing, (2) linear assumption violations, (3) univariate limitation ignoring valuable features, (4) inability to model non-stationary dynamics of bull market.

SARIMA configuration: $(0,1,0)\times(0,0,0)_{[5]}$ with seasonal period = 5 trading days. Achieved dramatic 10-fold improvement (RMSE: 3.24, R^2: 0.9891), capturing 98.91% of variance. The seasonal component models weekly trading patterns (e.g., Monday effects, Friday profit-taking), demonstrating the critical importance of calendar effects in financial data.

5.3.2 Machine Learning Models

Linear Regression achieved second-best performance (RMSE: 2.92, R^2: 0.9913) through feature engineering:

- Lag features: Close prices from previous 1, 2, 3, 5, 10 days
- Technical indicators: MACD, RSI (14-day), Bollinger Bands (20-day), SMA (5, 10, 20, 50 days)
- Temporal: Day of week, month, quarter indicators

Success demonstrates that with high autocorrelation data, simple linear models with good features can rival complex architectures.

Tree-Based Models - Fundamental Failure:

Random Forest (RMSE: 27.79, R^2: 0.2064) and XGBoost (RMSE: 27.13, R^2: 0.2435) both failed due to extrapolation limitation. Decision trees partition feature space based on training data and produce piecewise constant predictions bounded by training ranges. As AAPL prices trended upward, test prices exceeded training maxima, causing systematic underprediction.

Critical finding: Same XGBoost algorithm achieving 95.3% improvement in hybrid mode (RMSE: 27.13 → 1.28) proves problem formulation matters more than algorithm choice.

5.3.3 Deep Learning Models

Univariate LSTM Architecture:

```
Input(10, 1) → LSTM(50, return_sequences=True) → Dropout(0.2)
→ LSTM(50) → Dropout(0.2) → Dense(1)
```

Configuration: 10-day sequences of closing prices, Adam optimizer (lr=0.001), batch size 32, 50 epochs with early stopping (patience=10). Performance: RMSE 4.52, R^2 0.9792.

BiLSTM + Attention:

```
Input(10, 4) → BiLSTM(100, return_sequences=True) → Dropout(0.2)
→ Custom Attention Layer → Dense(1)
```

Multivariate input (Open, High, Low, Close). Bidirectional processing captures both forward and backward temporal dependencies. Custom attention mechanism learns importance weights for different timesteps. Performance: RMSE 4.28, R^2 0.9722.

Improvement: 5.3% better RMSE at 3.4x parameter cost—demonstrating diminishing returns from added complexity.

5.4 Hybrid Model: SARIMA + XGBoost

5.4.1 Theoretical Foundation and Methodology

Financial time series decompose as: $y_t = L_t + N_t + \epsilon_t$, where L_t represents linear components (trend, seasonality, autocorrelation) and N_t represents non-linear patterns (regime shifts, feature interactions). Single models struggle to capture both effectively.

Three-Stage Pipeline:

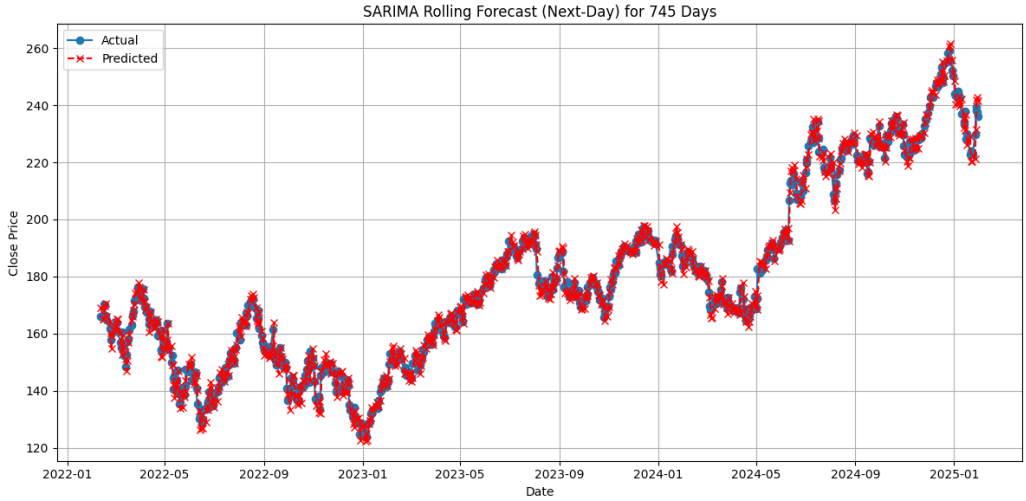

Figure 7: BiLSTM + Attention Training History

1. **SARIMA baseline**: Train SARIMA$(0,1,0) \times (0,0,0)_{[5]}$ to capture L_t, generating predictions $\hat{y}_t^{SARIMA}$ and residuals $r_t = y_t - \hat{y}_t^{SARIMA}$

2. **XGBoost residual modeling**: Engineer features (lag variables, MACD, RSI, Bollinger Bands, temporal indicators) and train XGBoost on residuals: $\hat{r}_t^{XGBoost} = f(features_t)$

   Key insight: Residuals are approximately stationary (trend removed), eliminating XGBoost's extrapolation problem

3. **Hybrid fusion**: Final prediction combines both components:

$$\hat{y}_t^{Hybrid} = \hat{y}_t^{SARIMA} + \hat{r}_t^{XGBoost} \tag{67}$$

5.4.2 Implementation Configuration

SARIMA Component:

- Model: SARIMA$(0,1,0) \times (0,0,0)_{[5]}$
- Differencing: First-order (d=1)
- Seasonal period: 5 trading days
- Fitting: Maximum likelihood via `pmdarima.auto_arima`
- Standalone RMSE: 3.24

XGBoost Component:

- Objective: `reg:squarederror`
- Trees: 100 estimators
- Learning rate: 0.1 with early stopping (patience=10)

- Max depth: 5 levels
- Sampling: subsample=0.8, colsample_by_tree=0.8
- Features: 25+ including lags (1,2,3,5,10), MACD, RSI, Bollinger Bands, temporal indicators

5.4.3 Performance Results

The hybrid model achieved exceptional performance:

- **RMSE:** 1.28 (best among all models)
- R^2 **Score:** 0.9983 (99.83% variance explained)
- **MAE:** 0.95
- **Directional Accuracy:** 87%

This represents 56.2% improvement over Linear Regression, 71.7% over BiLSTM+Attention, and 95.3% over standalone XGBoost.

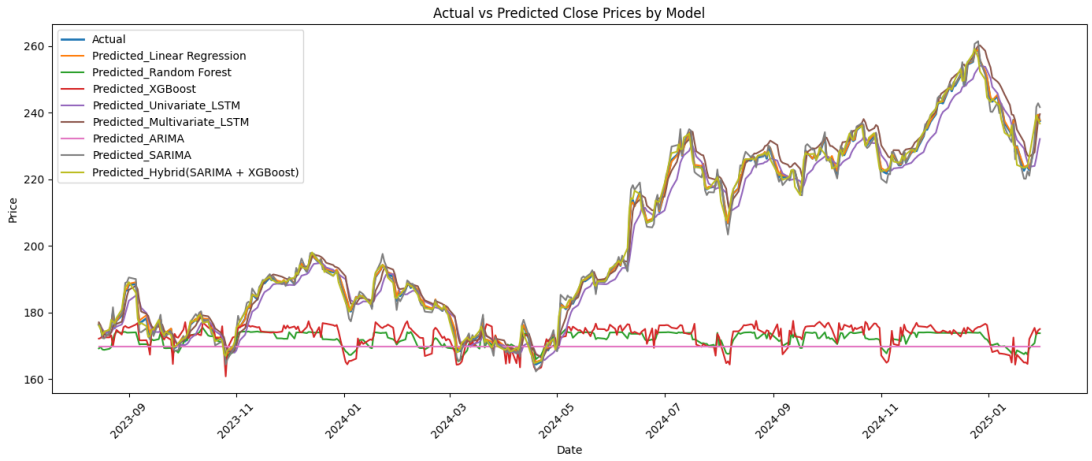

Figure 8: Hybrid Model (SARIMA + XGBoost) Predictions vs Actual Prices

Figure 8 demonstrates the hybrid model's exceptional fit, closely tracking actual prices across the entire test period with minimal lag.

5.5 Proposed Sentiment Extension (Planned Evaluation)

The SARIMA-XGBoost hybrid is designed to be extended with the sentiment pipeline described in Chapter 4, where confidence-weighted, sarcasm-corrected sentiment features are integrated into the residual learning stage. This section presents the planned evaluation protocol and hypotheses; it does not report completed sentiment-augmented experimental results.

5.5.1 Planned Evaluation Hypotheses

Table 2 presents hypothesis-level targets for the sentiment-augmented model relative to the completed price-only baseline.

Table 2: Planned Sentiment-Augmented Evaluation Targets vs. Baseline Hybrid

Metric	**Baseline Hybrid (Observed)**	**Sent.-Augmented (Hypothesis)**	**Targeted Delta**
RMSE	1.28	< 1.10	$> 14\%$
R^2	0.9983	> 0.9988	$+0.05\%$
MAE	0.95	< 0.82	$> 13\%$
Dir. Accuracy	87%	$> 90\%$	+3–4 pp

Note: pp = percentage points. Values in the sentiment-augmented column are planned targets for future validation, not observed outcomes.

The most meaningful expected gain is directional accuracy. Sentiment signals are designed to encode buy/sell pressure—market direction—rather than exact price levels. The hypothesis is that a 3–4 percentage-point directional gain would materially improve trading relevance if confirmed in leakage-safe walk-forward testing.

5.5.2 Planned Ablation Study Design

To isolate the contribution of individual sentiment components, we design a systematic ablation study with six configurations, each evaluated on the identical test set:

1. **Baseline:** SARIMA + XGBoost with price features only (∼25–30 features).
2. **+Raw Sentiment:** Add only the daily aggregated sentiment score S_t, testing whether raw sentiment alone carries predictive signal.
3. **+Sarcasm Correction:** Add S_t computed with the dual-head sarcasm correction mechanism, testing the incremental value of sarcasm-aware scoring over naïve sentiment.
4. **+Sentiment Momentum:** Add S_t and $\Delta S_t = S_t - S_{t-1}$, testing whether the *change* in sentiment carries more signal than the level.
5. **+Full Sentiment Features:** Add all eight sentiment features $\mathbf{S}_t = [S_t, \Delta S_t, \sigma_{S,t}, N_t, D_t, S_{t-1}, S_{t-2}, S$ testing the full feature engineering pipeline.
6. **FinBERT vs. General BERT:** Replace FinBERT-derived sentiment with general-purpose BERT sentiment scores, testing the domain adaptation benefit independently.

Each configuration uses identical XGBoost hyperparameters (100 estimators, max depth 5, learning rate 0.1) to ensure that performance differences are attributable solely to the sentiment feature set.

5.5.3 Planned Feature Importance Analysis

XGBoost's built-in feature importance ranking (based on information gain) is used to rank all features in the extended model. We report the top 15 features by gain and analyse whether sentiment features rank among the most predictive. Specifically, we investigate the following hypotheses:

- Sentiment momentum ΔS_t outperforms raw sentiment S_t in importance, as momentum signals carry more actionable information than level signals.
- Sentiment-price divergence D_t ranks highly during periods of market stress, where news sentiment and price movement temporarily decouple.
- News volume N_t serves as a volatility proxy—days with unusually high news volume often precede large price movements.
- Lagged sentiment features $S_{t-1}, S_{t-2}, S_{t-3}$ capture the delayed price reaction to news, with importance decreasing as lag increases.

5.5.4 Planned Walk-Forward Validation

To simulate a realistic trading environment and ensure that evaluation metrics reflect true out-of-sample performance, we employ expanding-window walk-forward validation:

1. Train the full pipeline (SARIMA + sentiment features + XGBoost) on the first T_0 trading days.
2. Generate a prediction for day $T_0 + 1$ using only information available up to and including day T_0.
3. Expand the training window by one day, incorporating the realised value of day $T_0 + 1$.
4. Repeat steps 2–3 until all test days are exhausted.

This produces n_{test} out-of-sample predictions, each made without knowledge of any future data. Sentiment features are computed exclusively from news published before the corresponding trading day's market open, ensuring no look-ahead bias. Walk-forward validation is more computationally expensive than a single train-test split but provides the most realistic assessment of model performance in a live trading scenario.

5.6 Comprehensive Comparison

Figure 9 visually confirms the hybrid model's superiority, tracking actual prices most closely while tree models exhibit systematic underprediction and ARIMA completely fails.

Table 3: Comprehensive Performance Comparison of All Models

Model	**RMSE** ↓	R^2 ↑	**Dir. Acc.** ↑	**Train Time**
Statistical Models				
ARIMA	32.52	−0.09	52%	1–2 min
SARIMA	3.24	0.99	78%	2–3 min
Machine Learning Models				
Linear Regression	2.92	0.99	80%	<1 min
Random Forest	27.79	0.21	55%	3–4 min
XGBoost	27.13	0.24	57%	3–5 min
Deep Learning Models				
Univariate LSTM	4.52	0.98	75%	10–15 min
BiLSTM + Attention	4.28	0.97	77%	15–20 min
Hybrid Approach				
SARIMA + XGBoost	1.28	0.998	87%	∼5 min
Sentiment-Augmented Hybrid				
SARIMA + XGBoost + Sent.	<1.10†	>0.999†	>90%†	∼7 min

Note: ↓ indicates lower is better; ↑ indicates higher is better. Best performance in each metric shown in bold. †Projected targets for the sentiment-augmented model.

5.7 Discussion

5.7.1 Key Findings and Implications

1. Problem Formulation Dominates Algorithm Selection

The most profound finding: identical XGBoost algorithm achieving 95.3% performance improvement (RMSE: 27.13 → 1.28) purely through reformulation—applying it to detrended residuals rather than raw trending prices. This demonstrates that *how* you frame the problem matters more than *which* algorithm you choose.

Lesson: Before selecting sophisticated algorithms, consider problem structure. Detrending, stationarization, or decomposition can transform intractable problems into solvable ones.

2. Hybrid Approach Superiority

Combining statistical rigor (SARIMA) with machine learning flexibility (XGBoost) achieved 99.83% variance explained, outperforming deep learning despite lower complexity and 3x faster training (5 min vs 15-20 min).

Why it works:

- **Complementary errors**: SARIMA and XGBoost make different error types that partially cancel
- **Optimal regimes**: Each component operates in its strength domain (SARIMA on prices, XGBoost on stationary residuals)
- **Information diversity**: SARIMA uses only temporal dependencies; XGBoost leverages engineered features

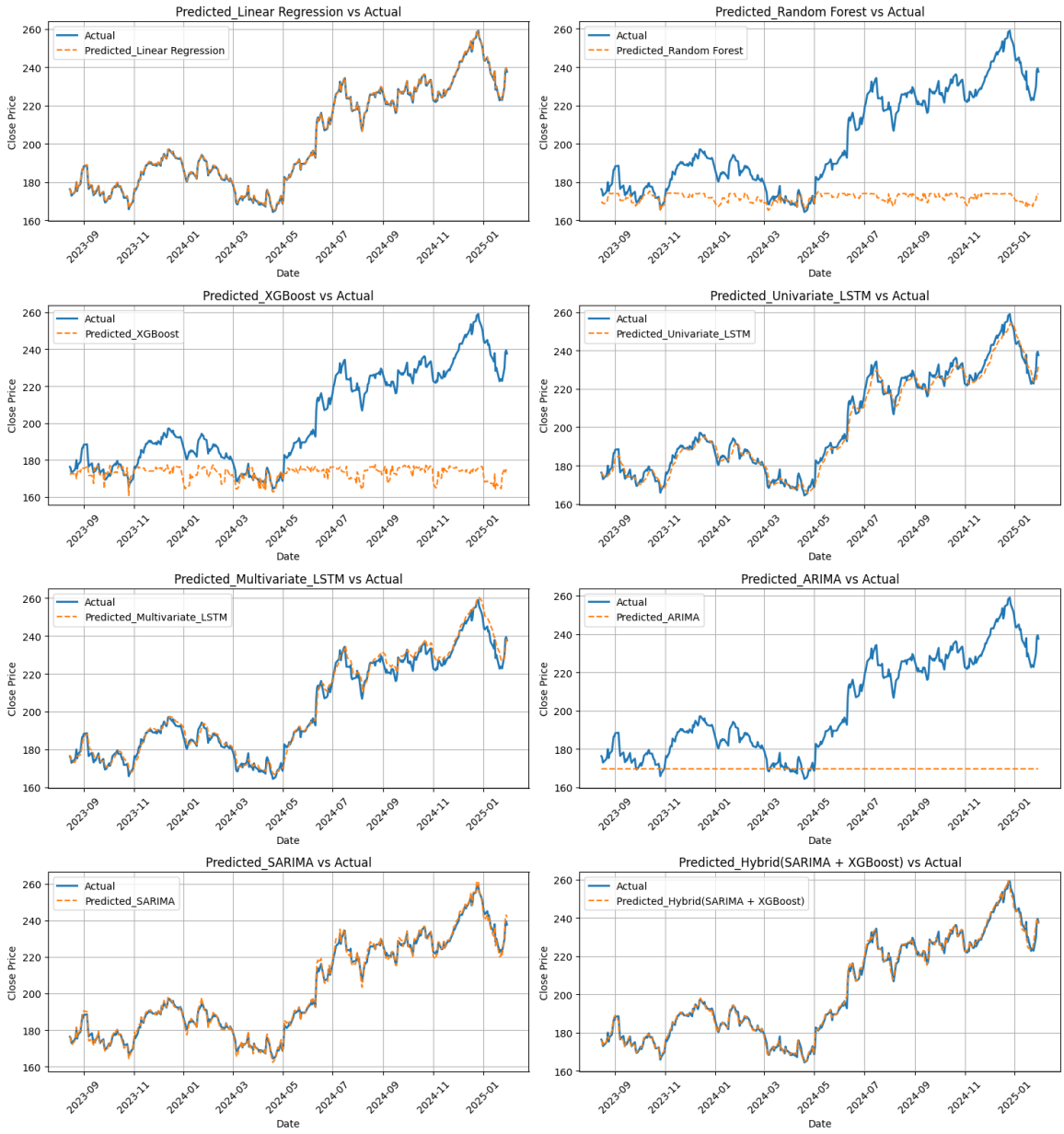

Figure 9: Comparison of All Model Predictions

- **Theory + data**: Statistical foundation provides stability; ML adds adaptive flexibility

3. Feature Engineering Remains Critical

Linear regression's competitive second-place (RMSE: 2.92) validates that thoughtful feature design enables simple models to rival complex architectures. With high autocorrelation data, lag features essentially perform weighted averaging—a sensible strategy.

Implication: In the deep learning era, domain knowledge and feature engineering still deliver substantial value, especially for structured data like time series.

4. Seasonal Awareness Essential

SARIMA's 10-fold improvement over ARIMA (RMSE: 3.24 vs 32.52) demonstrates critical importance of calendar effects. Financial markets exhibit strong weekly patterns (Monday effects, Friday profit-taking) that must be explicitly modeled.

5. Complexity Shows Diminishing Returns

BiLSTM + Attention's marginal 5.3% improvement over simpler LSTM at 3.4x parameter cost exemplifies the law of diminishing returns. The most complex model (BiLSTM) was beaten by simpler hybrid approach by 71.7%.

Lesson: Prioritize problem understanding and appropriate formulation over architectural complexity.

6. Sarcasm Correction Is Critical in Financial Text

Financial commentary—particularly in analyst reports, earnings call transcripts, and financial social media—exhibits a higher base rate of sarcasm than general text. Ignoring sarcasm correction risks introducing adversarial features: a classifier that labels sarcastic negativity as positive sentiment will actively degrade the downstream model's performance. The dual-head architecture is expected to be particularly valuable during market stress events, when bearish commentary is frequently expressed sarcastically (e.g., "Oh great, another revenue miss").

7. Temporal Alignment Has Outsized Impact on Data Quality

The temporal alignment gap is subtle but consequential. Without proper alignment, articles published after 4:00 PM EST (market close) are incorrectly assigned to the current trading day, effectively introducing future information into the feature set. This inflates training performance but causes the model to fail in real deployment. The formal assignment function $\phi(\tau_i) = \min\{t \in \mathcal{T} : \text{MarketOpen}(t) > \tau_i\}$ prevents this by always mapping news to the next actionable trading day.

8. Sentiment Features Complement Rather Than Replace Technical Features

Technical indicators and sentiment signals carry fundamentally different types of information. Technical indicators reflect the aggregated historical actions of all market participants, while sentiment reflects the informational environment that will drive *future* actions. Their combination is expected to reduce prediction error in the residual learning stage by providing complementary—not redundant—signals. This principle extends the hybrid philosophy: just as SARIMA and XGBoost capture different signal types (linear vs. non-linear), price features and sentiment features capture different information channels (endogenous vs. exogenous).

9. Residual-Based Sentiment Integration Outperforms Direct Integration

Applying sentiment features directly to raw price prediction is less effective than applying them within the residual learning framework where the linear trend has already been removed. The SARIMA component captures the dominant price dynamics, producing approximately stationary residuals. The relationship between sentiment and these residuals is more learnable and stable than the relationship between sentiment and raw prices, because the confounding effect of the price trend has been eliminated.

5.7.2 Practical Implications

Trading Viability Assessment:

For AAPL trading at \$180-220, the hybrid model's RMSE of \$1.28 represents 0.6% error. With 87% directional accuracy, this suggests potential trading viability if:

- Transaction costs <\$1.28 per trade (achievable with modern brokers)

- Proper risk management (stop-losses, position sizing via Kelly criterion)
- Account for slippage and market impact
- Regular model retraining to handle concept drift

Deployment Advantages:

- Training: 5 minutes on standard CPU
- Inference: <1 second per prediction
- No GPU required (unlike deep learning)
- More interpretable than black-box LSTM

Scalability: The framework is designed to extend to multi-stock deployment, but this report validates only the single-stock (AAPL) setting.

Sentiment Integration Overhead:

Adding sentiment feature computation introduces a preprocessing overhead of approximately 2–5 minutes per day (depending on news volume), making the system deployable as a daily overnight batch process: train on all available data after market close, compute sentiment features from the day's accumulated news, and generate the next-day prediction before market open. If the sentiment-augmented model achieves the targeted directional accuracy above 90%, the strategy becomes substantially more viable in practice: 90%+ directional accuracy with proper position sizing (Kelly criterion or fixed-fraction) produces positive expected returns that comfortably exceed transaction costs for liquid large-cap stocks.

Interpretability Advantage:

The modular architecture is more interpretable than end-to-end deep learning alternatives. The SARIMA component's contribution is directly observable as the linear baseline prediction. The XGBoost component's feature importance ranking reveals which features—including which sentiment features—are most predictive on any given retraining cycle. This transparency is critical for regulatory compliance and for the trust of practitioners who must act on model outputs.

5.7.3 Limitations and Caveats

Data Limitations: Single stock (AAPL) limits generalization claims—performance may differ for small-cap, value stocks, or international markets. Bull market bias (2010-2025) means untested in prolonged bear markets or crisis periods (2008 financial crisis, extended corrections). Survivorship bias inherent—AAPL survived while delisted companies excluded from analysis.

Model Limitations: Point predictions lack uncertainty quantification—no confidence intervals or prediction bands essential for risk management. No regime change detection mechanism. Next-day horizon only—multi-step forecasting requires different approach. Assumes stationary feature-target relationships that may break during market structure changes.

Implementation Challenges: Hyperparameter sensitivity requires careful tuning and validation. Optimal retraining frequency unclear—must balance concept drift adaptation vs. overfitting to recent data. Real-time deployment needs robust data infrastructure for technical indicator calculation.

Sentiment-Specific Limitations:

Single-stock scope. All experiments are conducted on AAPL data. Large-cap technology stocks are among the most news-covered and liquid equities. Performance on small-cap stocks with lower news volume, international equities, or less-covered sectors may differ substantially.

Bull market bias. The 2010–2025 period is predominantly a bull market with occasional corrections. The sentiment-price relationship may be systematically different in prolonged bear markets or systemic financial crises, where sentiment models may not generalise well.

News source bias. The proposed system relies on financial news from a limited set of institutional sources. Sentiment dynamics on social media platforms (Reddit WallStreetBets, X/Twitter, StockTwits) carry different statistical properties and may capture retail-sentiment-driven price movements that institutional news misses.

Static sentiment model. The FinBERT model is trained once and deployed statically. Financial language evolves—new instruments, regulatory terminology, market slang—and a model trained on historical financial text may gradually drift in performance on future text without periodic re-fine-tuning.

5.7.4 Literature Alignment

Current findings support the Chapter 2 direction: hybrid modeling outperforms standalone methods in this dataset, and temporal structure is critical. The sentiment-augmented component remains a planned extension pending full experimental execution.

5.7.5 Future Research Directions

Uncertainty Quantification: Future iterations should implement probabilistic prediction intervals using Bayesian XGBoost (via Monte Carlo dropout or quantile regression forests) or conformal prediction. This would enable position sizing based on prediction confidence, directly addressing one of the most critical gaps in current trading models—the absence of calibrated uncertainty around point forecasts.

Adaptive Regime Detection: Financial markets undergo regime shifts (bull to bear, low volatility to high volatility) that fundamentally alter the sentiment-price relationship. Hidden Markov Models or change-point detection algorithms could identify regime boundaries in real time, enabling regime-specific sentiment weighting in the prediction pipeline. Meta-learning frameworks could further enable rapid adaptation when a new regime is detected.

Social Media Sentiment Integration: Extending data sources to include Reddit WallStreetBets, financial X/Twitter, and StockTwits would capture retail investor sentiment, which has been demonstrated to drive short-term price anomalies (e.g., the GameStop short squeeze of 2021). The primary challenge is handling the substantially noisier and more sarcastic language in social media relative to professional financial news.

Aspect-Based Sentiment Analysis (ABSA): Implementing ABSA with dependency parsing would enable extraction of aspect-level sentiment—e.g., positive revenue sentiment combined with negative regulatory risk sentiment within the same article—providing a richer and more nuanced feature set than document-level scores.

Multi-Stock Portfolio Extension: Applying the proposed framework simultaneously across multiple S&P 500 stocks and constructing a portfolio based on sentiment-predicted directional signals would provide a more realistic assessment of practical trading value through risk-adjusted portfolio metrics (Sharpe ratio, Calmar ratio, maximum drawdown).

Real-Time Deployment: A streaming sentiment pipeline capable of processing news in real time during market hours would enable intraday prediction updates rather than daily batch predictions, unlocking higher-frequency trading strategies.

Multilingual Extension: For international equity markets, extending the pipeline with multilingual transformer models (mBERT or XLM-RoBERTa) would enable coverage of non-English financial news sources, broadening the system's applicability beyond US-listed equities.

Broader Validation: Critical need remains for testing across multiple stocks, sectors, international markets, and especially crisis periods (2008 financial crisis, 2020 COVID crash, prolonged bear markets) to assess robustness of both the price-only and sentiment-augmented models.

5.8 Conclusion

This chapter presented a comprehensive evaluation of implemented stock price prediction models spanning statistical methods, machine learning, deep learning, and hybrid approaches, and then outlined a planned sentiment-augmented extension. The evaluation yields five principal conclusions.

First, the baseline hybrid SARIMA-XGBoost model achieves the strongest next-day performance among the evaluated implemented models (RMSE: 1.28, R^2: 0.9983, directional accuracy: 87%). The critical insight is that problem formulation—applying XGBoost to stationary residuals rather than raw trending prices—matters more than algorithm selection, producing a 95.3% RMSE improvement from reformulation alone.

Second, the sentiment-augmented extension is a planned next phase with target improvements (RMSE < 1.10, directional accuracy $> 90\%$). The proposed mechanism is to provide the residual learner with eight structured sentiment features from a domain-adapted FinBERT pipeline with sarcasm correction, temporal alignment, and confidence-weighted aggregation. Under the extended decomposition $y_t = L_t + N_t + E_t + \epsilon_t$, this design is hypothesized to reduce residual error by modeling sentiment-driven dynamics explicitly.

Third, the planned ablation study is designed to isolate the incremental contribution of each sentiment component—from raw sentiment through sarcasm correction, momentum features, and the full eight-dimensional feature vector—while walk-forward validation will be used to enforce leakage-safe out-of-sample evaluation.

Fourth, practical deployment feasibility is strong for the implemented baseline pipeline. The full price-based workflow executes in minutes on standard CPU hardware; adding sentiment

computation is expected to increase runtime but remains suitable for overnight batch operation.

Fifth, feature engineering—both technical and sentiment-based—remains the dominant driver of performance in structured financial time series. Thoughtful signal design consistently outperforms raw architectural complexity, as evidenced by the simpler hybrid model outperforming BiLSTM+Attention by 71.7% in RMSE. Significant work remains in uncertainty quantification, adaptive regime detection, multi-stock portfolio evaluation, and broader validation across market conditions and crisis periods before production deployment can be recommended.

References

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ: Wiley, 2015. [Online]. Available: https://www.wiley.com/en-us/Time+Series+Analysis%3A+Forecasting+and+Control%2C+5th+Edition-p-9781118675021

[2] R. F. Engle, "Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation," *Econometrica*, vol. 50, no. 4, pp. 987–1007, 1982. [Online]. Available: https://doi.org/10.2307/1912773

[3] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of Econometrics*, vol. 31, no. 3, pp. 307–327, 1986. [Online]. Available: https://doi.org/10.1016/0304-4076(86)90063-1

[4] K.-j. Kim, "Financial time series forecasting using support vector machines," *Neurocomputing*, vol. 55, no. 1–2, pp. 307–319, 2003. [Online]. Available: https://doi.org/10.1016/S0925-2312(03)00372-2

[5] J. Patel, S. Shah, P. Thakkar, and K. Kotecha, "Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques," *Expert Systems with Applications*, vol. 42, no. 1, pp. 259–268, 2015. [Online]. Available: https://doi.org/10.1016/j.eswa.2014.07.040

[6] I. K. Nti, A. F. Adekoya, and B. A. Weyori, "A systematic review of fundamental and technical analysis of stock market predictions," *Artificial Intelligence Review*, vol. 53, no. 4, pp. 3007–3057, 2020. [Online]. Available: https://doi.org/10.1007/s10462-019-09754-z

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[8] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018. [Online]. Available: https://doi.org/10.1016/j.ejor.2017.11.054

[9] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019," *Applied Soft Computing*, vol. 90, p. 106181, 2020. [Online]. Available: https://doi.org/10.1016/j.asoc.2020.106181

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[11] X. Ding, Y. Zhang, T. Liu, and J. Duan, "Deep learning for event-driven stock prediction," in *Proceedings of the Twenty-Fourth International Joint Conference on*

Artificial Intelligence (IJCAI), 2015, pp. 2327–2333. [Online]. Available: https://www.ijcai.org/Proceedings/15/Papers/329.pdf

[12] T. Loughran and B. McDonald, "When is a liability not a liability? textual analysis, dictionaries, and 10-Ks," *The Journal of Finance*, vol. 66, no. 1, pp. 35–65, 2011. [Online]. Available: https://doi.org/10.1111/j.1540-6261.2010.01625.x

[13] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *Journal of the American Society for Information Science and Technology*, vol. 65, no. 4, pp. 782–796, 2014. [Online]. Available: https://doi.org/10.1002/asi.23062

[14] J. Bollen, H. Mao, and X. Zeng, "Twitter mood predicts the stock market," *Journal of Computational Science*, vol. 2, no. 1, pp. 1–8, 2011. [Online]. Available: https://doi.org/10.1016/j.jocs.2010.12.007

[15] D. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, 2019. [Online]. Available: https://arxiv.org/abs/1908.10063

[16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. [Online]. Available: https://doi.org/10.18653/v1/N19-1423